

PERANCANGAN SISTEM PEMANTAUAN CURAH HUJAN SEBAGAI PERINGATAN BANJIR BERBASIS INTERNET OF THINGS (IOT)

TUGAS AKHIR

**Karya tulis sebagai salah satu syarat
untuk memperoleh gelar Sarjana dari
Institut Teknologi Bandung**

Oleh

SAMUEL OBADIAH

NIM 10221076



**PROGRAM STUDI SARJANA FISIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
INSTITUT TEKNOLOGI BANDUNG
2025**

ABSTRAK

PERANCANGAN SISTEM PEMANTAUAN CURAH HUJAN SEBAGAI PERINGATAN BANJIR BERBASIS INTERNET OF THINGS (IOT)

Oleh
Samuel Obadiah
NIM: 102211076

Banjir merupakan bencana alam yang sering melanda masyarakat Indonesia. Salah satunya melanda Kecamatan Dayeukolot di Kabupaten Bandung yang disebabkan oleh hujan periodik yang menyebabkan meluapnya Sungai Citarum. Sehingga pengukuran curah hujan menjadi elemen yang penting dalam membangun sistem peringatan dini banjir. Penelitian ini mengajukan sistem berbasis IoT. Pengukuran curah hujan menggunakan sensor curah hujan RK400-07 dan pengukuran ketinggian permukaan sungai memanfaatkan *computer vision* dari BARDI IP Camera. Sebuah Mikrokontroler ESP32 digunakan untuk memproses hasil pengukuran-pengukuran tersebut dan mempersiapkan data tersebut untuk ditransmisikan ke sebuah *monitoring station* dengan modul LoRa SX1276. Hasil pengukuran tersebut lalu dimodelkan terhadap status kewaspadaan banjir di Kecamatan Dayeuhkolot dan Baleendah dengan menggunakan algoritma *machine learning linear regression* dan *decision tree*. Pengujian terhadap sensor curah hujan RK400-07 mendapati bahwa sensor dapat bekerja dengan baik. Pengujian modul LoRa SX1276 dilakukan pada kondisi lingkungan banyak gedung, banyak pohon, dan tanah lapang. Analisa keberpengaruhannya juga dilakukan dengan memodelkan data yang serupa, dan ditemukan bahwa hujan di daerah Dago dan Lembang paling mempengaruhi potensi banjir di Kecamatan Dayeuhkolot.

Kata kunci: Banjir, Curah Hujan, Internet of Things, Sistem Peringatan Dini

ABSTRACT

DESIGNING AN IOT-BASED RAINFALL MONITORING SYSTEM FOR FLOOD EARLY WARNING OF THE CITARUM RIVER

by

Samuel Obadiah

NIM: 10221076

Flooding is commonly occurring natural disaster in Indonesia and in the province of West Java. As of recent, the Dayeuhkolot district of Bandung Regency have experience flooding due to periodic rainfall causing the overflowing of the Citarum River. Therefore, the monitoring of rainfall at the Citarum river is essential in the early identification of potential flooding. This research proposes an IoT-based flood early warning system based on the measurement of rainfall using an RK400-07 rainfall sensor and computer vision by a BARDI IP Camera to monitor the river-level. An ESP32 microcontroller is used to process the measurements of the various sensors and prepare it for LoRa transmission to a *monitoring station*. Those measurements are then modeled toward flooding alert statuses for the Dayeuhkolot and Baleendah Districts using *linear regression* and *decision tree machine learning* algorithms. Testing of the rainfall sensor in order to determine the sensor's working capabilities and its found to be able to record rainfall well. In the urban environment which the rainfall measurement system is expected to operate, testing of the LoRa shows that it can transmit data up to 140 m. Feature Importance analysis is also done towards data of similar measurements, and it is found that Rainfall in Dago and Lembang are most influential to flooding events in the Dayeuhkolot District.

Key words: Early Warning, Internet of Things, Rainfall Measurement, Riverine Flooding.

PENGESAHAN

PERANCANGAN SISTEM PEMANTAUAN CURAH HUJAN SEBAGAI PERINGATAN BANJIR BERBASIS INTERNET OF THINGS (IOT)

Oleh

Samuel Obadiah

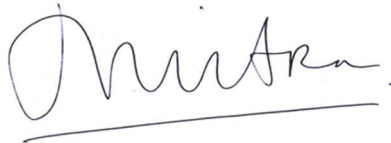
NIM 10221076

**Program Studi Sarjana Fisika
Fakultas Matematika dan Ilmu Pengetahuan Alam
Institut Teknologi Bandung**

Dosen Pembimbing Menyetujui Substansi dari Buku Tugas Akhir

Bandung, 03 September 2025

Dosen Pembimbing I,



Prof. Dr. Ing. Mitra Djamal
NIP 196005221985031002

Dosen Pembimbing II,



Dr. Nina Siti Aminah, S.Si., M.Si.
NIP 198304152012122004

Tim Penguji:

1. Dr. Maria Evita S. Si., M. Si.
2. Pipit Fitriani S. Si., Ph. D.

PEDOMAN PENGGUNAAN BUKU TUGAS AKHIR

Buku Tugas Akhir Sarjana ini tidak dipublikasikan, namun terdaftar dan tersedia di Perpustakaan Institut Teknologi Bandung. Buku ini dapat diakses umum, dengan ketentuan bahwa penulis memiliki hak cipta dengan mengikuti aturan HaKI yang berlaku di Institut Teknologi Bandung. Referensi kepustakaan diperkenankan dicatat, tetapi pengutipan atau peringkasan hanya dapat dilakukan seizin penulis, dan harus disertai dengan kebiasaan ilmiah untuk menyebutkan sumbernya.

Memperbanyak atau menerbitkan sebagian atau seluruh buku Tugas Akhir harus atas izin Program Studi Sarjana Fisika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Institut Teknologi Bandung.

(Lembar Persembahan)

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan yang telah memungkinkan saya untuk melakukan penelitian yang menjadi substansi dari tugas akhir saya. Saya juga berterimakasih kepada Pemerintah Provinsi Jawa Barat selaku pemberi dana melalui program Inovasi Pendanaan Pembangunan Kompetitif Hibah Kompetitif Tahun Anggaran 2024 bagi saya untuk melaksanakan tugas akhir saya. Juga saya ucapkan terimakasih kepada Bapak Ardiso Irwan Parebat selaku Kepala UP3BCH, Bapak Toriq Abdurahman selaku Staff IT Database UP3BCH, dan segenap jajaran Balai Besar Wilayah Sungai Citarum dalam menyediakan data yang membantu penyusunan substansi penelitian ini. Serta tidak lupa juga saya ucapkan terimakasih kepada Prof. Dr. Ing. Mitra Djamal dan Dr. Nina Siti Aminah, S. Si, M. Si., yang membimbing saya dalam mengerjakan penelitian ini.

Penelitian merupakan garda terdepan dalam mempelajari alam semesta yang begitu besar dan kompleks – untuk memperoleh pemahaman yang lebih dalam dan detail mengenai bagaimana sistem di alam bekerja. Penelitian dapat diadakan baik sebagai rangkaian penyelesaian studi seorang mahasiswa maupun sebagai pengejaran-sepanjang-karir seorang peneliti. Namun yang terutama bagi saya adalah penelitian harus mengedepankan kemanusiaan dan harus berujung pada memberi dampak baik bagi masyarakat. Walaupun setiap penelitian memiliki tujuan konkrit yang harus dicapai melalui metode-metode yang pada penelitian tersebut dijabarkan, namun implikasi dari tujuan tersebut sebaiknya membuat kehidupan umat manusia selangkah lebih maju dari sebelumnya.

Bencana alam merupakan peristiwa yang terjadi dari sejak dahulu dan di masa depan masih dimungkinkan untuk terjadi. Tetapi bukan berarti kita hanya dapat memaklumkan sebuah kejadian bencana alam begitu saja. Sebagai seorang insan akademis, sudah sebaiknya kita menjadi garda terdepan dalam mempelajari metode-metode terbagi dalam melakukan mitigasi bencana alam yang berdampak bagi sesama kita baik yang dekat maupun yang jauh – untuk meningkatkan kesadaran dan kesiapsiagaan bencana dan meminimalisasi kerusakan materi serta korban jiwa bilamana terjadi bencana.

Hal tersebut yang menjadi motivasi saya dalam melakukan penelitian ini. Karena bukanlah merupakan tanggungjawab kita yang lebih tahu dan lebih mengerti untuk memperjuangkan kesejahteraan mereka yang kurang tahu, kurang mengerti, dan –

dalam konteks fenomena bencana alam – kurang beruntung? Juga bukanlah hal tersebut juga berlaku dalam aspek kehidupan, dan bukan hanya mengenai penelitian? Terakhir, saya juga harus menyampaikan terimakasih saya kepada kedua orang tua saya, yang telah memperjuangkan saya hingga dapat berkuliah di Institut Teknologi Bandung dan menerima kesempatan untuk mengerjakan penelitian ini.

Bandung, 13 Agustus 2025

Samuel Obadiah

NIM 10221076

DAFTAR ISI

ABSTRAK.....	I
ABSTRACT	II
PENGESAHAN	III
KATA PENGANTAR.....	VI
DAFTAR ISI.....	VIII
DAFTAR TABEL	IX
DAFTAR GAMBAR.....	X
BAB I: PENDAHULUAN.....	1
I.1 Latar Belakang	1
I.2 Rumusan dan Batasan Masalah	3
I.3 Tujuan.....	5
I.4 Metodologi	5
I.5 Sistematika Penulisan	5
BAB II: TINJAUAN PUSTAKA.....	7
II.1 Sistem Peringatan Dini Banjir	7
II.2 Internet of Things.....	8
II.3 Sistem Pengukuran dalam Sensor Node.....	12
II.4 Analisis Data untuk Peringatan Dini Banjir	16
BAB III: METODOLOGI PENELITIAN	20
III.1 Perancangan dan Pengujian Sistem Pengukuran dalam Sensor Node.....	20
III.2 Perancangan Sistem Komunikasi dan Penyimpanan Data.....	23
III.3 Analisa Keberpengaruh data Curah Hujan terhadap Potensi Banjir	24
BAB IV: HASIL DAN ANALISIS	27
IV.1 Hasil Pengujian Sensor Node	27
IV.2 Hasil Analisa Keberpengaruh.....	32
BAB V: SIMPULAN DAN SARAN	36
V.1 Simpulan.....	36
V.2 Saran.....	37
DAFTAR PUSTAKA.....	38
LAMPIRAN A: PEMROGRAMAN	41

DAFTAR TABEL

Tabel II. 1. Tabel range nilai RSSI beserta keterangan kualitas sinyal yang diterima (Sallyna et al., 2020).....	12
Tabel II. 2. Tabel kategori curah hujan yang dapat diukur sensor curah hujan RK400-07 (RIKASENSOR, 2015).....	14
Tabel III. 1. Keterangan Pengukuran Curah Hujan dan Ketinggian Permukaan Sungai yang dimodelkan berdasarkan data dari BBWS Citarum (BBWS Citarum, 2025).....	24
Tabel III. 2. Keterangan status banjir berdasarkan ketinggian permukaan sungai pada PDA Dayeuhkolot (BBWS Citarum, 2025).....	26
Tabel IV. 1. Tabel nilai RSSI terhadap jarak antara dua modul LoRa SX1276 pada lingkungan banyak gedung	30
Tabel IV. 2. Tabel nilai RSSI terhadap jarak antara dua modul LoRa SX1276 pada lingkungan banyak pohon	31

DAFTAR GAMBAR

Gambar II. 1. (a) mekanisme kerja rainfall sensor RK400-07(RIKASENSOR, 2015). (b) rangkaian pengolah tipping rainfall sensor menjadi sinyal pulsa	14
Gambar II. 2. Contoh decision tree.....	17
Gambar III. 1. Arsitektur sistem peringatan dini banjir	20
Gambar III. 2. (a) arsitektur sistem pengujian sensor curah hujan RK400-07. (b) flowchart program pencatatan jumlah tipping pada pengujian sensor curah hujan RK400-07	22
Gambar III. 3. Arsitektur sensor node sistem peringatan dini banjir	23
Gambar III. 4. Denah aliran Sungai Citarum dan anak-anak sungainya (BBWS Citarum, 2025)	25
Gambar III. 5. Panel peringatan status banjir di Kecamatan Dayeuhkolot (BBWS Citarum, 2025)	25
Gambar IV. 1. Jumlah tipping yang dicatat dari sensor curah hujan RK400-07 pada tanggal 31 Mei 2025	27
Gambar IV. 2. Diagram box jumlah air yang terukur setiap satu kali tipping sensor hujan RK400-07	28
Gambar IV. 3. Perbandingan curah hujan yang diukur PCH Sarijadi dan sensor curah hujan pada tanggal 16 (a), 22 (b), dan 31 (c) Mei 2025.....	29
Gambar IV. 4. Penempatan modul LoRa SX1276 pada lingkungan banyak gedung dengan jarak: (a) 230 m, (b) 150 m, dan (c) 47 m	30
Gambar IV. 5. Penempatan LoRa SX1276 pada lingkungan tanah lapang dengan jarak 184 m. Nilai RSSI yang tercatat pada modul LoRa sebesar -121dB	30
Gambar IV. 6. Penempatan modul LoRa SX1276 pada lingkungan hutan dengan jarak: (a) 146 m dan (b) 18 m.....	31
Gambar IV. 7. Hasil pengukuran nilai RSSI terhadap jarak pada kondisi lingkungan banyak gedung, hutan, dan tanah lapang.....	31
Gambar IV. 8. Feature importance permodelan pengaruh curah hujan terhadap potensi banjir Kecamatan Dayeuhkolot menggunakan algoritma machine learning linear regression (atas) dan decision tree (bawah)	33

DAFTAR SINGKATAN

Singkatan	Arti
BBWS	Balai Besar Wilayah Sungai
CSS	<i>Chirp Spread Spectrum</i>
DAS	Daerah aliran sungai
FBP	<i>Flow-based Programming</i>
FEC	<i>Forward Error Correction</i>
GPS	<i>Global Positioning System</i>
IoT	<i>Internet of Things</i>
IP	<i>Internet Protocol</i>
MQTT	<i>Message Queueing Telemetry Transport</i>
PCH	Pos Curah Hujan
PDA	Pos Duga Air
PWM	<i>Pulse Width Modulation</i>
RMSE	<i>Root Mean Squared Error</i>
RTSP	<i>Real Time Streaming Protocol</i>
SoA	<i>Service-oriented Architecture</i>
UDP	<i>User Datagram Protocol</i>
WiFi	<i>Wireless Fidelity</i>
YOLO	<i>You Only Look Once</i>

BAB I

PENDAHULUAN

I.1 Latar Belakang

Banjir merupakan ancaman bencana alam yang sering melanda masyarakat Indonesia. Badan Nasional Penanggulangan Bencana (BNPB) mencatat terjadinya 1255 kejadian bencana banjir pada tahun 2023 di seluruh Indonesia – menjadikannya bencana alam ketiga paling sering terjadi setelah bencana kebakaran hutan dan lahan serta bencana cuaca ekstrem. Di Provinsi Jawa Barat, data yang sama mencatat terjadinya 107 kali banjir, yang menjadikan Jawa Barat sebagai provinsi dengan jumlah kejadian bencana banjir terbanyak kedua setelah provinsi Sumatera Utara pada tahun 2023 (BNPB, 2024). Bencana alam banjir dapat menimbulkan kerugian besar bagi masyarakat yang terdampak banjir. Studi yang dilakukan Jayantara menunjukkan bahwa kerugian yang ditimbulkan bencana banjir di Kabupaten Bandung ditaksir mencapai sebelas ribu triliun rupiah, dengan kerugian terbesar dialami sektor rumah tangga (Jayantara, 2020). Hal tersebut menunjukkan bahwa banjir adalah ancaman yang besar terhadap negara dan masyarakat baik pada skala nasional, maupun pada Provinsi Jawa barat dan Kabupaten Bandung terkhususnya.

Hujan merupakan penyebab utama banjir. Namun mekanisme hujan yang menyebabkan banjir tersebut dapat diklasifikasi sebagai banjir bandang, banjir hujan-singkat, dan banjir hujan-lebat. Dimana banjir hujan-lebat menyebabkan 65% kejadian bencana banjir secara global pada tahun 2003 (Douben, 2006). Banjir juga dapat diasosiasikan dengan pola cuaca yang lebih spesifik seperti monsun dan siklon, serta faktor lingkungan lainnya – baik alami maupun buatan manusia. Sehingga sulit untuk mengatribusikan suatu penyebab tertentu terhadap sebuah kejadian banjir. Namun permodelan yang dilakukan terhadap banjir di Jerman menunjukkan bahwa daerah pesisir sungai berpotensi mengalami kerusakan paling besar akibat banjir (Kreibich & Dimitrova, 2010).

Profil geografis yang sama juga ditemukan di Kabupaten Bandung yang terletak pada Cekungan Bandung yang merupakan daerah aliran sungai (DAS) Citarum, Cisangkuy, dan Cikapundung (Simanjuntak et al., 2023; Sitorus et al., 2021). Hujan yang periodik dan berkelanjutan ditemukan menyebabkan terjadinya banjir jangka

panjang yang melanda daerah tersebut. Salah satunya adalah banjir berkelanjutan pada tahun 2019 yang terjadi selama ± 30 hari di Kecamatan Baleendah. Banjir yang disebabkan oleh aliran sungai dan hujan berkelanjutan menandakan bahwa untuk mempelajari, memodelkan, dan memprediksi banjir pada daerah tersebut, diperlukan pengukuran secara real-time terhadap ketinggian air dan curah hujan pada/di Sungai Citarum sebagai bentuk pemantauan terhadap potensi banjir.

Pemantauan terhadap potensi banjir umumnya dilakukan melalui berbagai infrastruktur pengendali banjir yang terletak pada sungai tersebut – yang dapat tersebar sepanjang aliran sungai (Krzhozhanovskaya et al., 2011). Kesulitan dari implementasi tersebut adalah membuat sistem yang dapat mengelola banyaknya dan beragamnya data yang diukur dari berbagai sistem tersebut sehingga mudah dipahami agar pengelolaan banjir dapat dilaksanakan dengan tepat, serta agar masyarakat sekitar dapat memahami potensi banjir mendatang – yang diketahui dapat meningkatkan ketahanan masyarakat tersebut terhadap bencana banjir (Tadrist et al., 2022). Sistem pengukuran dalam Internet of Things (IoT) dapat menjadi solusi yang unggul dan efektif dalam membangun sistem prediksi banjir yang tepat guna demi kebutuhan pengendalian banjir serta dapat diakses oleh masyarakat dengan mudah. Perkembangan dalam IoT yang meliputi sistem komunikasi efisien, jaringan sensor pintar, komputasi *cloud*, dan analisa prediktif telah menambah nilai potensi IoT sehingga daya gunanya dalam bidang industri, kesehatan, dan pembangunan tidak dapat lagi diabaikan (Li et al., 2015). Dari perkembangan dan implementasi IoT tersebut, telah tersedia berbagai platform dan perangkat yang tersedia secara umum yang dapat digunakan untuk membangun sistem dengan IoT untuk berbagai aplikasi – seperti untuk sistem deteksi banjir.

Pemanfaatan IoT untuk sistem deteksi banjir telah diteliti sebelumnya, sebagian besar memanfaatkan water level sensor modul berbasis kapasitansi atau sensor ultrasonik untuk menentukan ketinggian air (A'Ziziyyah et al., 2022; Hadi et al., 2020; Muzakky et al., 2019; Windiastik et al., 2019). Walaupun termasuk solusi yang murah, kemampuan dan ketahanannya pada pengujian skala besar di lapangan belum teruji. Penelitian-penelitian yang sama juga memanfaatkan *sensor node* dengan mikrokontroler ESP yang terhubung dengan platform IoT BYLINK. Penggunaan mikrokontroler ESP – yang merupakan mikrokontroler dengan konektivitas dengan WiFi – sudah tepat untuk implementasinya sebagai perangkat *sensor node* dalam IoT. Namun penggunaan platform BYLINK memiliki keterbatasan dalam *scalability* sensor-sensor *node* untuk sistem peringatan dini

banjir yang efektif digunakan secara nyata. Juga pada penelitian-penelitian yang sebelumnya dilakukan hanya memanfaatkan pengukuran ketinggian air sungai untuk memprediksi banjir, dengan demikian tidak memanfaatkan pengukuran curah hujan dalam menentukan prediksi tersebut. Penelitian oleh A'Ziziyyah juga mempelajari implementasi algoritma untuk mengidentifikasi potensi banjir (A'Ziziyyah et al., 2022), namun dalam implementasi sistem peringatan dini banjir dibutuhkan permodelan dari pengukuran yang dilakukan *sensor node* berdasarkan data-data kurun waktu (*time series*). Juga pengukuran yang melibatkan sistem dengan IoT diuntungkan oleh pengukuran pada banyak lokasi dan pengelolaan pengukuran tersebut dengan mudah. Sehingga potensi banjir yang menjadi parameter dalam peringatan dini banjir dapat dimodelkan bukan hanya berdasarkan pengukuran tunggal suatu lokasi pada aliran sungai, namun sebagai model dari sistem sungai tersebut dan bagaimana sistem tersebut dipengaruhi oleh hujan pada berbagai lokasi sepanjang aliran sungai tersebut.

Sehingga penelitian tugas akhir ini bertujuan untuk merancang, membangun, dan mengimplementasikan secara nyata sistem peringatan dini banjir berbasis IoT pada Sungai Citarum pada kecamatan Bojongsoang, Dayeuhkolot, dan Baleendah. Sistem peringatan dini banjir yang dirancang memanfaatkan *sensor node* dengan mikrokontroler ESP32 untuk memproses dan meneruskan data yang diterima dari *rain gauge* RK400-07, sensor DHT11, dan modul Global Positioning System (GPS) Ublox NEO 7M menuju sebuah *server* MQTT Mosquitto. *Server* MQTT dikelola, diprogram, dan dijalankan dengan Node-RED. Pengukuran ketinggian air dilakukan dengan memanfaatkan *computer vision* Roboflow untuk membaca pengukuran ketinggian air dari *rain gauge* oleh kamera IP. Program untuk menjalankan *computer vision* terintegrasi dengan *server* MQTT dengan dijalankan dengan Node-RED. Data-data yang diukur oleh *sensor node* ditransmisikan menuju stasiun pemantauan dengan memanfaatkan teknologi komunikasi LoRa pada modul LoRa SX1276. Data-data tersebut dikumpulkan dan pada Python digunakan algoritma *linear regression* dan *decision tree* untuk membuat model dan menganalisa keberpengaruhannya hujan pada berbagai daerah di Bandung terhadap potensi banjir di Kecamatan Dayeuhkolot.

I.2 Rumusan dan Batasan Masalah

Sehingga berdasarkan latar belakang yang telah dijabarkan, maka dapat dijabarkan rumusan masalah yang meliputi poin-poin pertanyaan berikut.

1. Bagaimana rancangan sistem komunikasi dengan RS485 yang dapat

mentransmisikan data dari Rainfall Sensor RK400-07 ke Mikrokontroller ESP32?

2. Bagaimana rancangan jaringan MQTT yang dapat mentransmisikan data dari Mikrokontroller ESP32 ke *server* MQTT?
3. Bagaimana rancangan sistem komunikasi yang memanfaatkan teknologi LoRa agar dapat mentransmisikan data dari *sensor node* menuju stasiun pengendali?
4. Bagaimana rancangan sistem penyimpanan data dengan InfluxDB dan visualisasi data dengan Grafana agar dapat menyimpan data pengukuran dari *sensor node* sebagai data *time series*?
5. Bagaimana keberpengaruhan curah hujan pada berbagai daerah di Bandung terhadap potensi banjir di Kecamatan Dayeuhkolot dan Baleendah?

Adapun batasan masalah dan asumsi yang digunakan dalam merancang dan membangun sistem prediksi banjir pada penelitian tugas akhir ini ditunjukkan poin-poin berikut.

1. Pengujian perangkat LoRa yang digunakan dilakukan pada lingkungan Kampus Ganesha Institut Teknologi Bandung (ITB), di lapangan Sarana Olahraga (Saraga) ITB, dan di Hutan Kota Babakan Siliwangi.
2. Data yang dikumpulkan *sensor node* adalah data lokasi menggunakan modul GPS Ublox NEO 7M, temperatur dan kelembaban udara menggunakan Sensor DHT11, ketinggian air yang ditunjukan pada stream BARDI IP Camera yang diterjemahkan dengan *Computer vision* Roboflow, dan curah hujan yang dikur menggunakan Rainfall Sensor RK400-07.
3. Error pada pengukuran oleh sensor-sensor serta ketidaksesuaian dengan spesifikasi pada komponen-komponen yang digunakan – baik akibat usia, cacat produksi, maupun dari penggunaan komponen tersebut –, serta variabilitas spasial dari pengukuran curah hujan dianggap minim sehingga dapat diabaikan.
4. Data yang digunakan untuk membangun model *machine learning* yang digunakan untuk analisa keberpengaruhan potensi banjir adalah data curah hujan dan ketinggian permukaan sungai berdasarkan pengukuran dengan alat-alat yang tercantum pada Tabel III.1 yang dikumpulkan dari tanggal 1 Maret 2025 pukul 00:00 hingga tanggal 7 Juli 2025 pukul 10:00.
5. Digunakan algoritma *machine learning linear regression* untuk mempelajari daerah-daerah yang berpotensi menyebabkan banjir di Kecamatan Dayeuhkolot serta algoritma *decision tree* untuk memetakan pengukuran-pengukuran curah hujan tersebut terhadap potensi banjir di Kecamatan Dayeuhkolot. Dianggap selama pengukuran yang dimodelkan selalu terdapat aliran air pada Sungai

Citarum walaupun tidak terdapat hujan dan hujan di berbagai daerah di Bandung selalu dapat menambah ketinggian permukaan Sungai Citarum di Kecamatan Dayeuhkolot.

I.3 Tujuan

Sehingga tujuan dari penelitian tugas akhir ini adalah sebagai berikut.

1. Merancang *sensor node* bertenaga surya yang dapat mengukur curah hujan dan ketinggian air Sungai Citarum.
2. Merancang sistem komunikasi data antara perangkat dalam *sensor node* serta sistem komunikasi data jarak jauh antara *sensor node* dengan stasiun pengendali.
3. Membuat model dengan *machine learning* yang dapat digunakan untuk menganalisa keberpengaruh hujan pada berbagai daerah di Bandung sebagai upaya peringatan dini banjir di Kecamatan Dayeuhkolot.

I.4 Metodologi

Metode yang digunakan untuk mencapai tujuan yang telah dijabarkan pada penelitian tugas akhir ini diantaranya sebagai berikut.

1. Studi Pustaka
Melakukan studi pustaka terhadap perangkat keras dan lunak beserta cara kerjanya berdasarkan buku, review jurnal, *datasheet* perangkat, serta dokumen lainnya untuk mengevaluasi sistem yang dirancang.
2. Eksperimen
Mengadakan eksperimen berupa pengujian terhadap sistem peringatan dini banjir beserta sub-sistemnya untuk menentukan kemampuan dari sistem yang dibangun.
3. Komputasi
Mengadakan komputasi untuk mendeskripsikan berbagai pengukuran sebagai data *time series* dan memodelkan data untuk menentukan keberpengaruhannya terhadap potensi banjir.

I.5 Sistematika Penulisan

Sebagaimana diulas pada bab ini, Bab II akan membahas mengenai perangkat keras dan lunak yang digunakan dalam sistem peringatan dini banjir yang dirancang beserta cara kerja dan konsep fisis pada perangkat-perangkat tersebut. Bab III

menjabarkan mengenai metodologi yang digunakan dalam membangun sistem peringatan dini banjir pada penelitian tugas akhir ini. Bab IV akan meninjau sistem peringatan dini banjir yang dibangun, model *machine learning* yang dihasilkan, serta kemampuan sub-sistem pada *sensor node*. Bab V membahas kesimpulan yang dapat diambil mengenai sistem peringatan dini banjir yang dibangun beserta saran untuk tinjauan lebih lanjut.

BAB II

TINJAUAN PUSTAKA

Perangkat deteksi dini yang dibangun pada penelitian tugas akhir ini memanfaatkan berbagai konsep dasar. Pengertian mengenai konsep-konsep tersebut serta konvensi mengenai pemanfaatannya dibahas pada tinjauan pustaka berikut.

II.1 Sistem Peringatan Dini Banjir

Sebuah sistem peringatan dini banjir didefinisikan oleh organisasi UN Office for Disaster Risk Reduction (UNDRR) sebagai sistem yang dapat memantau, menganalisa, dan memprediksi potensi banjir secara terintegrasi dan berperan dalam meningkatkan kesiapan individu, masyarakat dan pemerintah untuk menghadapi bencana tersebut dalam waktu yang cukup (Perera et al., 2019). Dalam sistem tersebut, pengukuran sistem fisis yang menjadi moda dalam identifikasi dan prediksi potensi banjir – seperti curah hujan dan pola hidrologi, dilakukan dalam jaringan dan saling terhubung secara *real-time*.

Sehingga sistem dengan Internet of Things (IoT) – topik yang mana akan dibahas pada subab selanjutnya – memiliki manfaat yang sangat besar yang menjadikan implementasinya integral untuk mencapai tujuan dari sebuah sistem peringatan dini banjir. Sebuah sistem peringatan dini banjir dengan IoT dapat melakukan pengukuran dan mengelola infrastruktur pengendali banjir yang masing-masing dapat berjarak kilometer secara real-time, dan dapat diintegrasikan dengan teknologi lainnya – seperti pencitraan satelit dan simulasi banjir – untuk mensuplemen kemampuan sistem dalam mengidentifikasi potensi banjir sebagai upaa peringatan dini banjir (Krzyszhanovskaya et al., 2011).

Juga telah dilakukan penelitian-penelitian mengenai implementasi sistem peringatan dini banjir secara sederhana. Tujuan dari penelitian tersebut adalah untuk mendemonstrasikan dalam skala lab beberapa sub-sistem dalam sistem pemantauan sungai dan bagaimana sebuah peringatan banjir dapat diberikan berdasarkan pengukuran tersebut. Penelitian oleh Hadi dan Muzakky merupakan dua contoh dari demonstrasi tersebut, dimana sebuah sistem pemantauan ketinggian permukaan sungai dibangun dengan memanfaatkan modul-modul sensor yang tersedia untuk mikrokontroller ESP8266. Kedua penelitian tersebut juga

memanfaatkan BYLINK sebagai platform untuk mengintegrasikan hasil pengukuran dengan sensor tersebut dengan aplikasi pemantauan data sebagai sistem dengan IoT (Hadi et al., 2020; Muzakky et al., 2019). Penelitian oleh Hadi juga mendemonstrasikan integrasi dengan sistem pengendali banjir sederhana dengan menggunakan katup solenoid untuk mengatur ketinggian permukaan sungai berdasarkan sebuah status potensi banjir berdasarkan ketinggian permukaan sungai yang terukur (Hadi et al., 2020).

Upaya untuk mengimplementasikan sebuah sistem peringatan dini banjir juga telah dipelajari pada penelitian yang dilaksanakan oleh A'ziziyahh – dengan memanfaatkan data yang dikumpulkan sistem pengukuran yang dibangun untuk menentukan potensi banjir pada sebuah titik pengukuran di waktu mendatang (A'Ziziyyah et al., 2022). Hal tersebut dicapai dengan memanfaatkan algoritma Naive Bayes dan K-Nearest Neighbours (K-NN) untuk menghasilkan data prediksi berdasarkan data pengukuran pada saat itu. A'ziziyahh menemukan bahwa algoritma K-NN menghasilkan hasil prediksi yang lebih akurat dibandingkan dengan bila memanfaatkan algoritma Naive Bayes. Selebihnya sistem pengukuran yang dibuat pada penelitian tersebut sama dengan penelitian-penelitian yang sebelumnya dibahas – memanfaatkan mikrokontroler ESP8266 untuk mentransmisikan data untuk ditunjukkan pada BYLINK.

II.2 Internet of Things

Internet of Things (IoT) didefinisikan sebagai sebuah jaringan dinamis yang dapat mengelola dirinya sehingga berbagai perangkat dan sistem dengan berbagai protokol komunikasi dapat saling terhubung untuk mentransmisikan data. Pada IoT, perangkat dalam IoT – disebut sebagai things – memiliki konektivitas terhadap jaringan internet dan dapat memanfaatkan data atau proses baik yang disediakan oleh perangkat lain, maupun proses yang sepenuhnya terjadi secara virtual di Cloud (Li et al., 2015). Setiap perangkat memiliki identitas dalam jaringan. Sehingga setiap data yang ditransmisikan dan/atau disimpan pada jaringan dapat dipasangkan dengan perangkat yang memberikan data tersebut, yang memudahkan pengelolaan data dalam jaringan yang berjumlah banyak dan bervariasi.

Dalam membangun sistem dengan IoT yang baik, perlu diperhatikan bagaimana berbagai perangkat, proses, dan pengguna saling berinteraksi untuk mencapai tujuan dibangunnya sistem tersebut. Sistem-sistem dalam IoT dengan service-oriented architecture (SoA) secara umum memiliki empat komponen berikut:

sensing layer, network layer, service layer, dan interfaces layer. Sensing layer merupakan komponen-komponen yang dapat mengukur besaran fisis atau menerima input fisik – umumnya berupa perangkat keras sensor. Network layer merupakan sistem yang mentransmisikan data dalam jaringan pada IoT. Service layer merupakan platform yang mengelola dan mengolah data yang terkumpul pada sistem IoT. Interface layer merupakan moda visualisasi, komunikasi, dan interaksi data oleh pengguna maupun sistem lain (Li et al., 2015).

II.2.1 Jaringan MQTT

Message Queuing Telemetry Transport (MQTT) merupakan protokol internet yang mentransmisikan data antara perangkat (things) yang diatur untuk mengirimkan atau menerima data tersebut. Dalam mengirimkan data, digunakan *broker* atau *server* sebagai penyelia antara perangkat yang mengirimkan (*publish*) data dan perangkat yang menerima (*subscribe*) data. Dalam IoT, bisa terdapat banyak perangkat yang melakukan pola *publish-subscribe* tersebut. *Broker* membedakan data-data yang ditransmisikan tersebut menurut *topic* yang dimiliki oleh setiap data. *Topic* diatur oleh setiap perangkat publisher dan perangkat subscriber hanya akan menerima data dengan *topic* tertentu (Hillar, 2017).

Keunggulan dari MQTT adalah kemampuan mentransmisikan banyak data dengan cepat dan efisien. MQTT bekerja di atas Internet Protocol (IP) sehingga setiap perangkat yang dapat terhubung ke internet dapat mengakses MQTT. Setiap perangkat yang mengakses sebuah jaringan MQTT disebut sebagai MQTT client, dan untuk mengakses jaringan MQTT tersebut MQTT client harus mengirimkan kepada *server* MQTT informasi mengenai *username* dan *password* dari koneksi MQTT tersebut, serta *client id* dari perangkat yang akan mengakses jaringan MQTT. Bila informasi mengenai koneksi MQTT sudah sesuai dengan *server*, maka *server* akan menerima perangkat tersebut dan perangkat tersebut dapat mengirimkan atau menerima data pada jaringan MQTT.

II.2.2 Node-RED untuk pemrograman *server* MQTT

Node-RED merupakan aplikasi *flow-based programming* (FBP) yang digunakan untuk memvisualisasikan, memprogram, dan menjalankan proses yang terjadi pada *server* MQTT. Pada Node-RED, setiap proses direpresentasikan sebagai sebuah node yang mengandung program yang menjalankan sebuah proses. Keunggulan Node-RED adalah sebagai aplikasi FBP, Node-RED mudah diimplementasikan untuk proses pada MQTT yang sederhana, memiliki banyak library yang memperluas potensi kegunaannya, dan sebagai program berbasis web, dapat

diakses oleh berbagai perangkat yang terhubung pada jaringan MQTT maupun dijalankan pada cloud (Hagino, 2021).

II.2.3 Modulasi Gelombang

Modulasi gelombang merupakan metode untuk menyimpan informasi berupa sinyal dalam gelombang. Dikarenakan gelombang memenuhi prinsip superposisi, maka sebuah gelombang dapat dideduksi menurut komponen frekuensinya. Pada proses modulasi, komponen frekuensi tersebut dapat dimanipulasi amplitudonya sesuai dengan informasi yang akan ditransmisikan. Metode tersebut disebut sebagai modulasi amplitudo. Untuk membawa informasi yang ditransmisikan tersebut, digunakan gelombang *carrier* yang menjadi pusat dimana komponen harmonik dari gelombang *modulator* – gelombang menjadi informasi yang dimodulasi – akan berada disekitar frekuensi gelombang *carrier* tersebut. Komponen harmonik dengan frekuensi lebih rendah dari gelombang *carrier* disebut sebagai *lower side band* dan komponen harmonik dengan frekuensi lebih tinggi dari gelombang *carrier* disebut sebagai *upper side band*. Beda frekuensi antara frekuensi terendah *lower side band* dengan frekuensi tertinggi *upper side band* disebut sebagai bandwidth (Crawford, 1968).

Selain dengan memanipulasi amplitudo frekuensi disekitar frekuensi gelombang *carrier*, juga dapat dimanipulasi frekuensi dari gelombang *carrier* itu sendiri. Metode modulasi tersebut disebut sebagai *frequency modulation*. Pada *frequency modulation*, gelombang modulator menentukan laju frekuensi dari gelombang yang dihasilkan, dengan gelombang *carrier* menjadi gelombang hasil yang tidak dipengaruhi gelombang *modulator* (Torres et al., 2001). *Phase modulation* juga merupakan modulasi dengan memanipulasi gelombang *carrier*, namun informasi pada gelombang hasilnya dikandung pada beda fasa terhadap gelombang *carrier* (Crawford, 1968). *Frequency multiplexing* merupakan metode untuk menyimpan dan mentransmisikan dua informasi berbeda pada satu gelombang *carrier*. Hal tersebut dicapai dengan menempatkan informasi pada *range* frekuensi (*band*) yang berbeda agar komponen harmonik kedua informasi tidak bertumpukan.

Metode lain untuk menyimpan informasi pada gelombang adalah melalui modulasi digital, pada modulasi digital, informasi direpresentasikan sebagai logika *high* dan *low*. Salah satu bentuk dari modulasi digital adalah *pulse width modulation* (PWM). Untuk mengekstrak informasi (demodulasi) dengan PWM, gelombang didekati dengan gelombang *ramp* atau segitiga. Fasa awal pada gelombang *ramp* memulai dengan logika *high* pada gelombang hasil demodulasi. Ketika gelombang *ramp*

berpotongan dengan gelombang yang didemodulasi, maka gelombang hasil akan berubah logika menjadi *low*. Perubahan pada gelombang yang didemodulasi akan mempengaruhi lebar sinyal kotak (*pulse*) pada gelombang hasil. PWM merupakan metode yang umum digunakan dalam mengatur tegangan pada perangkat elektronika, dan dapat ditemukan pada konverter DC-DC (Sun, 2012).

II.2.4 LoRa dan jaringan LoRaWAN untuk Transmisi Data Jarak Jauh

Long Range (LoRa) merupakan teknologi Low Power Wide Area Network (LPWAN) yang dikembangkan oleh Semtech. LoRa memanfaatkan modulasi yang serupa dengan Chirp Spread Spectrum (CSS) untuk mentransmisikan sinyal yang dapat menempuh jarak yang jauh tanpa terpengaruh oleh faktor lingkungan seperti interferensi, efek doppler, dan atenuasi jarak. Sebuah *chirp* merupakan sinyal dengan frekuensi yang membesar menurut waktu. Pada LoRa, frekuensi awal sebuah chirp mengandung data yang ditransmisikan, dan *chirp* dibedakan menurut frekuensi awalnya (Gkotsiopoulos et al., 2021).

Terdapat parameter-parameter yang dapat diatur untuk menentukan karakteristik dari LoRa, diantaranya adalah *carrier frequency*, spreading factor, bandwidth, dan coding rate (Bor et al., 2016). *Carrier frequency* merupakan frekuensi gelombang pembawa sinyal yang ditransmisikan. LoRa bekerja pada frekuensi dibawah frekuensi seluler dan harus mematuhi regulasi dimana perangkat dengan LoRa tersebut dijalankan. Indonesia membatasi frekuensi penggunaan perangkat dengan LoRa pada frekuensi 920-923 MHz (KOMINFO, 2019). Spreading factor merupakan rasio antara *chip rate* – jumlah pulsa-per-detik suatu data diproses – terhadap symbol rate – jumlah perubahan signifikan per-detik dari suatu data (Bor et al., 2016). Spreading factor menentukan T_{sample} , yaitu seberapa lama frekuensi chirp meningkat pada Bandwidth dari LoRa. Bandwidth merupakan range frekuensi modulasi chirp. LoRa di Indonesia diatur untuk memiliki bandwidth selebar 125KHz (Nashiruddin & Hidayati, 2019). LoRa dengan spreading factor kecil memiliki frekuensi yang meningkat dengan lebih cepat, sehingga lebih banyak data yang dapat ditransmisikan pada suatu periode waktu tertentu. Namun LoRa dengan spreading factor kecil lebih rentan terhadap interferensi dan atenuasi (Gkotsiopoulos et al., 2021).

LoRa didesain untuk memiliki resiliensi terhadap interferensi lingkungan salah satunya dengan memiliki beberapa bit yang mengulang informasi bit sebelumnya. Proses tersebut disebut sebagai Forward Error Correction (FEC), dan proses tersebut dilakukan untuk mengurangi error yang ditimbulkan oleh interferensi dari

lingkungan (Nashiruddin & Hidayati, 2019). *Coding rate* merupakan rasio jumlah bit yang berisi data asli terhadap jumlah bit yang ditransmisikan. Menambah *coding rate* dapat meningkatkan ketahanan data terhadap interferensi lingkungan, namun juga meningkatkan lama waktu data untuk ditransmisikan (Sallyna et al., 2020). RSSI merupakan salah satu parameter yang menunjukkan kualitas sinyal yang diterima LoRa. RSSI menunjukkan daya terima keseluruhan sinyal yang ditransmisikan pada sebuah channel LoRa. RSSI bernilai negatif dengan semakin kecil nilainya, semakin buruk sinyal yang diterima LoRa (Sallyna et al., 2020). Berikut merupakan tabel yang memetakan suatu *range* nilai RSSI terhadap kualitas sinyal yang diterima.

Tabel II. 1. Tabel range nilai RSSI beserta keterangan kualitas sinyal yang diterima (Sallyna et al., 2020).

RSSI (dBm)	Keterangan
-30 s/d -60	Sangat kuat. Jarak pemancar dan penerima sangat dekat.
-60 s/d -90	Sangat baik. Cakupan dekat.
-90 s/d -105	Baik. Terdapat beberapa data yang tidak diterima.
-105 s/d -115	Buruk. Dapat menerima tetapi sering drop-out
-115 s/d -120	Sangat buruk. Sinyal lemah data sering hilang

Keunggulan dari LoRa diantaranya adalah transmisi data dengan LoRa lebih efisien dibandingkan dengan transmisi data dengan teknologi seluler. Aksesibilitas LoRa juga mudah dikarenakan komunikasi LoRa juga dilakukan pada frekuensi dibawah 1GHz, sehingga tidak terdapat batasan lisensi untuk menggunakan LoRa. LoRa dapat digunakan sebagai metode komunikasi antar perangkat (machine-to-machine) dengan menghubungkan perangkat tersebut dengan transceiver LoRa (Prakosa et al., 2021).

Semtech SX1276 merupakan *chip* LoRa yang diproduksi oleh perusahaan Semtech. LoRa tersebut dapat diprogram untuk bekerja pada frekuensi 137-1020 MHz dengan spreading factor yang dapat diatur pada nilai 6-12 dan coding rate pada nilai 4/5, 4/6, 4/7, serta 4/8. Lora tersebut memiliki dynamic range RSSI sebesar 127 dB dan diperuntukkan untuk aplikasi baik oleh perorangan maupun dalam industri (SEMTECH, 2020).

II.3 Sistem Pengukuran dalam *Sensor Node*

Sensor node berperan sebagai things dalam sistem dengan IoT yang dibangun. Di dalam *sensor node*, terdapat berbagai sensor yang mengirimkan data dengan

memanfaatkan berbagai protokol komunikasi. Berikut adalah perangkat-perangkat yang diutilisasikan untuk melakukan pengukuran, mengelola, dan mentransmisikan data pada *sensor node*.

II.3.1 Mikrokontroller ESP32

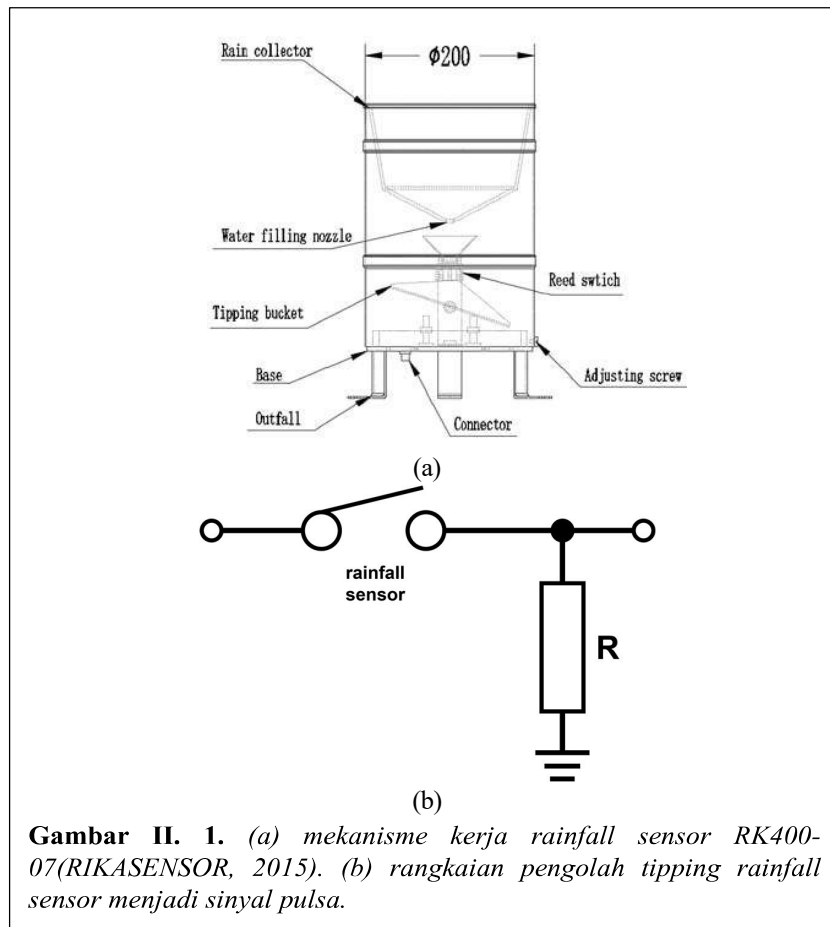
ESP32 merupakan *chip* mikrokontroller yang diproduksi oleh Espressif Systems. Pada ESP32 terdapat mikroprosesor *dual-core* Tensilica Xtensa LX6. ESP32 dapat terhubung ke jaringan *wireless fidelity* (WiFi) 2.4GHz dan Bluetooth 4.2. Sehingga ESP32 umum digunakan dalam membangun sistem IoT untuk berbagai aplikasi, terkhususnya untuk aplikasi yang membutuhkan komputasi yang intensif atau yang proses yang perlu dilakukan dengan cepat. ESP32 tersedia dalam bentuk *chip* maupun dalam bentuk modul *development board* yang disediakan oleh Espressif Systems maupun oleh perusahaan *third-party* (Hercog et al., 2023; Kurniawan, 2019).

Pemrograman ESP32 dapat dilakukan dengan dua cara, dengan Arduino IDE atau dengan software development kit (SDK) yang disediakan Espressif Systems. Pemrograman dengan Arduino menggunakan bahasa pemrograman C++ dan ESP32 diprogram selayaknya sebuah mikrokontroller Arduino (Hercog et al., 2023). Keunggulan dari pemrograman dengan Arduino IDE adalah tersedianya library yang mempermudah pemrograman ESP32 untuk menerima data dari berbagai sensor dan menterjemahkan data berbagai protokol komunikasi. Namun tidak dimungkinkan untuk mengatur proses pada ESP32 secara detail dengan pemrograman dengan Arduino IDE selayaknya pemrograman dengan SDK ESP32. Pemrograman melalui SDK ESP32 juga memanfaatkan bahasa pemrograman C++, namun penulisan program dilakukan di Python dengan dimungkinkan oleh library khusus yang disediakan Espressif Systems (Kurniawan, 2019). Keunggulan dari pemrograman metode ini adalah dimungkinkannya optimasi lebih lanjut terhadap proses pada ESP32.

II.3.2 Rainfall Sensor RK400-07

Rainfall Sensor RK400-07 merupakan sensor curah hujan yang diproduksi oleh Hunan Rika Electronic Tech Co., Ltd. Pada sensor tersebut terdapat *reed switch* yang mengirimkan sinyal pulsa berdasarkan air hujan yang melalui sensor tersebut. Satu sinyal pulsa dikalibrasi untuk menandakan curah hujan sebesar 1mm sesuai dengan mekanisme *tipping bucket* di dalam sensor tersebut. Ketika *tipping bucket* telah penuh maka sensor akan mengalirkan air dan menutup reed switch. Reed switch lalu memberikan sinyal pulsa yang dicatat oleh *data logger* pada sensor

(RIKASENSOR, 2015). Berikut merupakan diagram yang menunjukkan mekanisme kerja sensor tersebut.



Gambar II. 1. (a) mekanisme kerja rainfall sensor RK400-07(RIKASENSOR, 2015). (b) rangkaian pengolah tipping rainfall sensor menjadi sinyal pulsa.

Sensor curah hujan tersebut hanya dapat mengukur curah hujan maksimum delapan milimeter dalam satu menit (RIKASENSOR, 2015). Berikut merupakan keterangan mengenai kategori intensitas curah hujan yang dapat diukur oleh sensor curah hujan RK400-07 yang diberikan oleh Rikasensor.

Tabel II. 2. Tabel kategori curah hujan yang dapat diukur sensor curah hujan RK400-07 (RIKASENSOR, 2015).

Skala	Curah hujan (mm hari)
light rain	1-9.9
moderate rain	10-24.9
heavy rain	25-49.9
rainstorm	50-99.9
torrential rain	100-200
super-torrential rain	>200

II.3.3 *Computer vision* untuk Pengukuran Ketinggian Air Sungai dengan Roboflow

Computer vision merupakan proses komputasi untuk mendeduksi gambar menurut komponen-komponen fisis yang terkandung pada gambar tersebut seperti warna, bentuk, dan ukuran. Pemanfaatan *computer vision* sangat luas, dengan contoh aplikasinya diantaranya pada *optical character recognition*, mobil *self-driving*, dan *motion capture*. Proses dalam *computer vision* merupakan proses inversi data gambar menjadi data numerik yang dapat diproses oleh komputer (Szeliski, 2022). Dalam melakukan proses inversi tersebut, dapat digunakan algoritma *machine learning* bagi komputer untuk mengenali elemen-elemen pada gambar. Salah satu algoritma yang dapat digunakan adalah YOLO.

YOLO merupakan singkatan dari You Only Look Once, dan merupakan algoritma *machine learning* yang efektif dalam mendeduksi obyek pada gambar secara real-time. YOLO mendeteksi obyek pada gambar dan meletakkannya di dalam sebuah bounding box. Kemudian YOLO mendeduksi obyek tersebut. Proses deduksi yang dilakukan dapat dibagi menjadi tiga kategori proses: backbone, neck, dan head. Backbone merujuk pada proses mendeduksi fitur-fitur sederhana obyek seperti bentuk dan tekstur. Fitur-fitur yang dideteksi pada backbone lalu saling dicocokkan pada neck. Head lalu melakukan prediksi mengenai obyek berdasarkan fitur-fitur yang telah dicocokkan tersebut. YOLO lalu menggunakan algoritma Non-maximum Supression (NMS) untuk menentukan obyek yang dianggap penting dengan mengabaikan bounding box yang saling bertabrakan (Terven et al., 2023).

II.3.4 Protokol RTSP untuk Transmisi Gambar dalam *Computer vision*

Video yang direkam oleh kamera IP dapat diakses oleh perangkat yang pada jaringan yang sama melalui Real Time Streaming Protocol (RTSP). RTSP dikembangkan oleh Real Networks, Netscape Communications, dan Universitas Columbia sebagai metode untuk mentransmisikan stream multimedia – seperti audio dan video – yang kini digunakan sebagai standar dalam industri. Streaming merupakan proses mencacah data multimedia menjadi paket data yang dapat ditransmisikan secara real-time (Liu, 1997). RTSP menerima stream media dan menjaga koneksi agar stream tersebut tetap aktif untuk diakses oleh perangkat-perangkat yang mengakses stream tersebut. RTSP tidak terkait dengan koneksi transport-level dan dapat bekerja diatas User Datagram Protocol (UDP) – yang memungkinkan streaming video dengan lebih cepat. RTSP memiliki beberapa

parameter koneksi yang sama dengan HTTP, yang memudahkan setup sebuah stream RTSP. Sehingga dimungkinkan untuk memulai akses stream RTSP dari web (Schulzrinne et al., 1998).

II.4 Analisis Data untuk Peringatan Dini Banjir

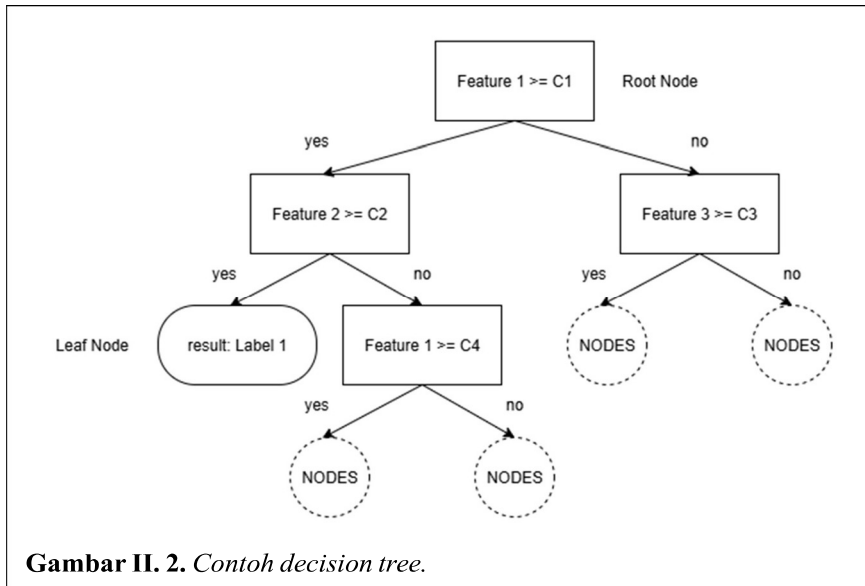
II.4.1 Time Series dan Analisa Time Series dengan Grafana

Data yang terukur pada suatu interval yang telah ditentukan sebelumnya merupakan sebuah *time series*. Pada database *time series*, data-data dari berbagai pengukuran dikelompokkan menjadi suatu paket data yang sama, sehingga suatu data pada pengukuran tertentu dapat diakses dengan lebih cepat. Data-data dari pengukuran yang sejenis yang diukur pada interval waktu tertentu juga dikelompokkan sebagai sebuah *measurement*. Analisa mengenai data *time series* dapat menunjukkan perubahan suatu nilai yang terukur terhadap waktu, konteksnya mengenai sistem dimana nilai tersebut diukur, serta dapat digunakan untuk memprediksi nilai di masa depan (Naqvi et al., 2017).

Data *time series* terdapat pada IoT dalam bentuk nilai dari pengukuran sensor setiap waktu tertentu. Sehingga monitoring dan pengendalian terhadap things pada IoT dapat dilakukan dengan efisien. Data yang dikumpulkan juga memungkinkan analisis real-time terhadap sistem-sistem pada IoT, yang juga memungkinkan pengambilan keputusan dengan cepat dan tepat berdasarkan model mengenai sistem tersebut yang dibuat dari data *time series*.

II.4.2 Feature Importance dalam Analisa Keberpengaruhan Pengukuran dalam Jaringan

Untuk meningkatkan ketepatan-gunaan hasil pengukuran dari *sensor node* dari berbagai lokasi pengukuran sepanjang aliran sungai, diperlukan pemahaman terhadap relasi masing-masing hasil pengukuran terhadap potensi bencana banjir. Dalam memodelkan relasi tersebut, dapat digunakan *machine learning* untuk mendeskripsikan bagaimana pengaruh pengukuran curah hujan terhadap potensi banjir. Parameter potensi banjir dapat berupa data kontinu berdasarkan ketinggian permukaan sungai, atau berupa klasifikasi status kewaspadaan banjir (A'Ziziyah et al., 2022; BBWS Citarum, 2025). Sehingga digunakan model *machine learning* untuk mengkategorikan potensi banjir menurut pengaruh curah hujan pada berbagai lokasi.



Salah satu metode permodelan dengan *machine learning* yang dapat digunakan adalah algoritma *decision tree*. Sebuah decision tree merupakan serangkaian pertanyaan yang yang dihasilkan dari algoritma yang memasang parameter-parameter (fitur) dengan hasil yang didapatkan bila parameter-parameter tersebut dipenuhi (label). Misalkan Decision tree sebagaimana pada Gambar II.4 diatas. Berdasarkan data yang dipelajari, sekumpulan fitur pertama-tama diuji menurut nilai Feature 1 – yang disebut sebagai akar atau *root node*. Bilamana nilai Feature 1 lebih besar atau sama dengan C1, maka sekumpulan fitur tersebut akan kembali diuji menurut pertanyaan pada percabangan ‘yes’. Bila tidak, maka sekumpulan fitur tersebut akan diuji menurut pertanyaan pada percabangan ‘no’. Sistematis pertanyaan tersebut dihasilkan dari pemisahan data yang dipelajari menurut struktur pertanyaan tersebut. Kumpulan fitur tersebut akan terus diuji hingga mencapai sebuah leaf node – yaitu node yang dari pemisahan data yang dipelajari menghasilkan satu kategori label (mencapai *purity*), atau dihentikan menurut parameter-parameter model yang diatur.

Terdapat beberapa parameter yang digunakan sebagai batasan dalam pembelajaran dengan algoritma *decision tree*. Dua diantaranya adalah *branch limit* dan *min samples*. Parameter *branch limit* menunjukkan percabangan maksimum yang dilakukan dalam proses pembelajaran dengan menggunakan algoritma *decision tree*. Bilamana telah dilakukan *splitting* sebanyak *branch limit* namun data pada node tersebut belum mencapai *purity*, pecabangan tersebut tetap akan diklasifikasi menurut kategori label yang paling sering muncul. Hal yang sama juga akan

dilakukan bila jumlah rangkaian fitur-label yang dipelajari pada sebuah percabangan lebih sedikit dari parameter *min samples*.

Selain dengan algoritma *decision tree*, pengukuran curah hujan dan relasinya terhadap potensi banjir juga dapat dimodelkan sebagai persamaan linear (Maulud & Abdulazeez, 2020), dimana nilai target didapatkan melalui kombinasi linear setiap fitur yang masing-masing mempengaruhi nilai target sebesar koefisien dari masing-masing fitur. Sehingga dapat juga digunakan algoritma *linear regression* untuk mengatribusikan besar koefisien yang tepat bagi masing-masing fitur agar didapatkan model yang menghasilkan nilai target yang tepat menurut persamaan berikut:

$$y = c + m_1x_1 + m_2x_2 + \dots + m_ix_i + s \quad (1)$$

Dengan y merupakan nilai label yang menjadi target untuk dimodelkan, x merupakan fitur yang menjadi vektor *input* pada model tersebut, m merupakan koefisien dari setiap vektor *input*, c merupakan konstanta dari model tersebut, dan s merupakan error dari model tersebut. Dalam membuat model dengan algoritma *linear regression*, diusahakan untuk meminimalisasi error dari model yang dihasilkan sehingga besar error dapat diabaikan dari model yang didapatkan.

Untuk mendapatkan model dari data yang dipelajari, dilakukan proses inversi dari kumpulan data tersebut sebagai matriks fitur, matriks label, dan matriks koefisien sebagaimana pada persamaan berikut:

$$M = (X^T X)^{-1} X^T Y \quad (2)$$

Dengan M merupakan matriks koefisien dari model yang dibuat, X merupakan matriks fitur-fitur yang dipelajari, dan Y merupakan matriks label bilamana nilai dari fitur-fitur yang bersebelahan dipenuhi (Maulud & Abdulazeez, 2020). Pada algoritma *linear regression* yang digunakan dapat ditetapkan beberapa batasan atau parameter dari model yang dibuat. Koefisien dari setiap fitur dapat dipaksa sebagai nilai yang tidak lebih kecil dari nol (*force positive*) dan garis regresi yang dibuat juga dapat ditetapkan untuk bernilai nol pada saat nilai semua fitur nol (*slope intercept*).

Pada permodelan dengan memanfaatkan kedua algoritma tersebut, fitur-fitur mempengaruhi nilai target yang dihasilkan. *Feature importance* merupakan

parameter yang menjelaskan seberapa berpengaruh sebuah fitur terhadap target dari model yang dihasilkan. *Feature importance* dapat digunakan baik untuk menjelaskan data (Altmann et al., 2010) maupun sebagai metode untuk menseleksi fitur pada model sebagai metode untuk mengefisiensikan permodelan data (Chaudhary et al., 2016). Pada implementasinya dalam sistem peringatan dini banjir, dilakukan identifikasi terhadap pengukuran curah hujan yang paling mempengaruhi potensi banjir pada suatu titik sepanjang sungai melalui analisa *feature importance* guna menyusun sistem peringatan dini banjir yang efektif berdasarkan semua pengukuran curah hujan yang dilakukan.

Nilai *feature importance* direpresentasikan pada model *linear regression* pada koefisien dari setiap fitur pada model tersebut. Sementara pada model *decision tree*, nilai *feature importance* didapatkan dari persamaan berikut:

$$I_G = \sum_c \Delta i(r) \quad (3)$$

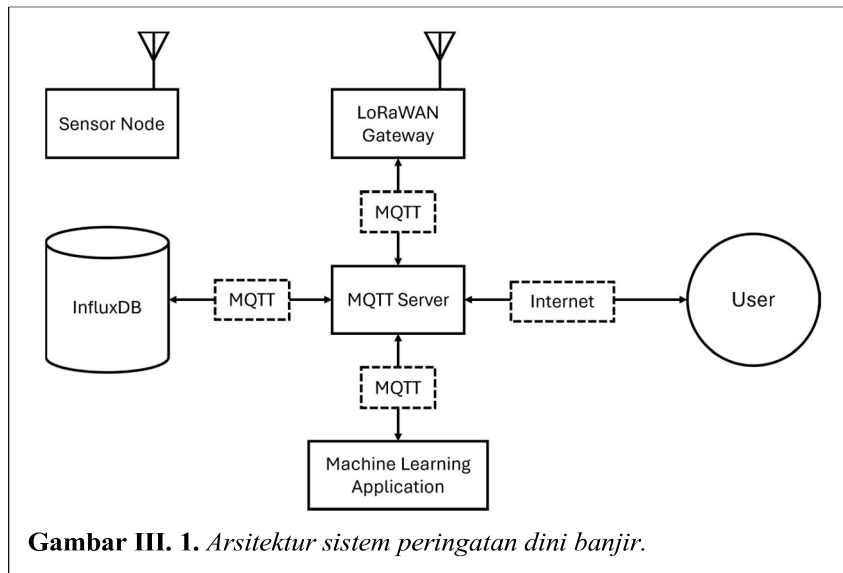
$$\Delta i(r) = i(r) - p_{yes}i(r_{yes}) - p_{no}i(r_{no}) \quad (4)$$

Nilai *feature importance* dari sebuah fitur (I_G) didapatkan dari menjumlahkan semua perubahan *purity* dari node yang menggunakan fitur tersebut untuk *splitting* – sebagaimana ditunjukkan pada persamaan 3. Perubahan *purity* sendiri dirumuskan pada persamaan 4 sebagai nilai *purity* ternormalisasi ($i(r)$) pada node tersebut dikurangi dengan jumlah *purity* ternormalisasi pada node yang dihasilkan melalui *splitting* menggunakan node tersebut yang kembali dinormalisasi menurut jumlah data pada node tersebut (p).

BAB III

METODOLOGI PENELITIAN

Dari tinjauan Pustaka yang telah dilakukan, meninjau tujuan dari penelitian tugas akhir ini, yaitu untuk membangun sistem monitoring ketinggian air dan curah hujan pada/di Sungai Citarum untuk peringatan dini banjir, maka dijabarkan metodologi penelitian yang mencakup perancangan sistem pengukuran dalam *sensor node*, perancangan sistem komunikasi dan penyimpanan data, perancangan sistem penyuplai daya dengan panel surya, dan perancangan model *time series* untuk prediksi banjir. Berikut merupakan arsitektur dari rancangan sistem peringatan dini banjir yang dibangun pada penelitian tugas akhir ini.



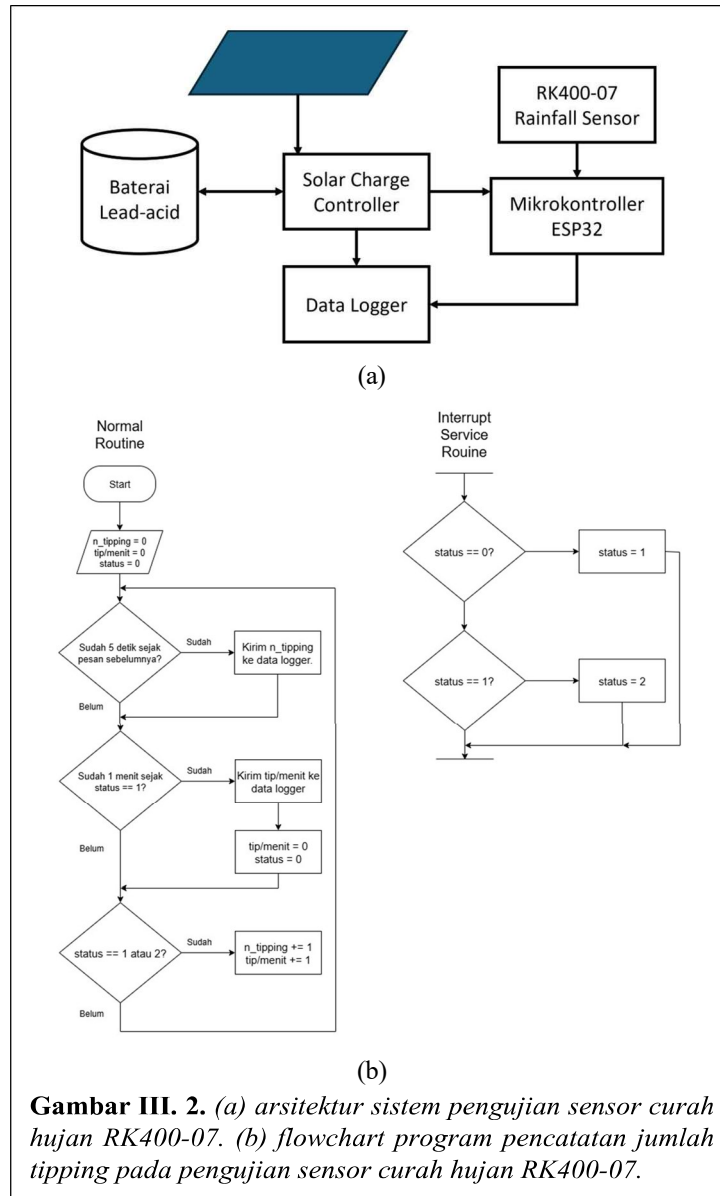
Dapat diperhatikan pada diagram diatas bahwa sistem peringatan dini banjir yang dirancang pada penelitian ini meliputi sensor node – yang sub-sistemnya akan dibahas pada bagian-bagian berikutnya – dan monitoring station. Pada monitoring station terdapat sistem *gateway* LoRa yang mengolah sinyal LoRa dari *sensor node*

III.1 Perancangan dan Pengujian Sistem Pengukuran dalam *Sensor node*

Pengukuran curah hujan dilakukan dengan *rain gauge* RK400-07 yang diatur dengan mikrokontroller ESP32. *Rain gauge* RK400-07 yang bila dirangkai

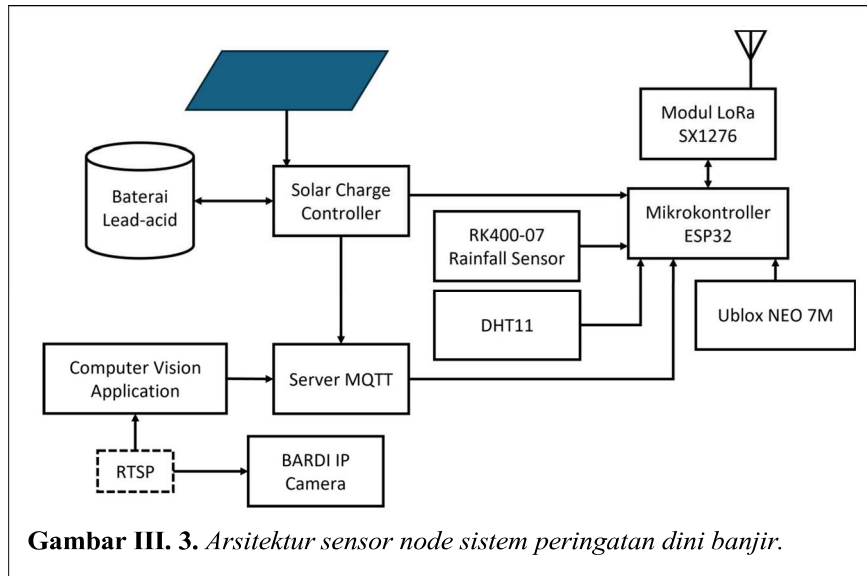
sebagaimana pada gambar dibawah, memberikan sinyal pulsa yang dikirimkan ke *server* Raspberry Pi 4 untuk diproses dan disimpan. Pemrograman mikrokontroler ESP32 dilakukan dengan perangkat lunak Arduino IDE, yang menerjemahkan bahasa pemrograman C++ menjadi program yang dapat dijalankan mikrokontroler ESP32.

Pengujian sistem pengukuran curah hujan dilakukan dengan membandingkan jumlah *tipping* per jam yang diberikan sensor hujan RK400-07 dengan data curah hujan dari pos curah hujan (PCH) yang diterbitkan oleh BBWS Citarum. Pengujian dilakukan untuk mengkalibrasi satu kali *tipping* dari rain gauge dengan sejumlah curah hujan dalam mm. Berikut merupakan arsitektur sistem pengukuran beserta flowchart program pada mikrokontroler untuk mengukur dan mencatat curah hujan.



Pengukuran ketinggian air dilakukan dengan memanfaatkan *computer vision* untuk menerjemahkan ketinggian air yang terlihat pada water level gauge menjadi data integer yang menunjukkan ketinggian air dalam cm. Digunakan Python untuk menjalankan program *computer vision* yang YOLOv8, dengan program python tersebut dijalankan pada Raspberry Pi 4 sebagai *server* MQTT di *sensor node*.

Pada *sensor node* juga dilakukan pengukuran suhu dan kelembaban udara dengan sensor DHT11 beserta pencatatan lokasi dimana *sensor node* diletakkan dengan modul GPS Ublox NEO 7M. Dengan demikian, berikut merupakan arsitektur dari sistem pengukuran pada *sensor node*.



Dapat diperhatikan pada flowchart diatas bahwa sistem pengujian sensor curah hujan melakukan penghitungan jumlah *tipping* setiap satu menit dan jumlah *tipping* total. Pengukuran jumlah *tipping* per menit dilakukan untuk menguji kemampuan sensor hujan terhadap jumlah curah hujan maksimum yang dapat diukur sebagaimana keterangan yang diberikan oleh produsen sensor tersebut pada Tabel II.2. Juga dilakukan pengukuran total jumlah *tipping* yang dapat diolah sebagai curah hujan pada sebuah periode sepuluh menit sebagaimana pada pengukuran curah hujan yang dilakukan PCH BBWS Citarum – yang akan dijelaskan pada subbab berikutnya.

III.2 Perancangan Sistem Komunikasi dan Penyimpanan Data

Data pengukuran curah hujan dan ketinggian air pada mikrokontroller dikirim ke *server* MQTT pada stasiun monitoring dengan menggunakan protokol MQTT yang disediakan *server* Raspberry Pi yang menjalankan program mosquitto. Data pada broker diolah untuk disimpan pada database InfluxDB. Pengolahan dan query data pada broker dilakukan dengan program web Node Red. Model *machine learning* disimpan dan dijalankan pada *server* untuk menghasilkan data prediksi. Data pengukuran dan data prediksi dikirimkan ke pusat kendali secara machine-to-machine dengan memanfaatkan modul transceiver LoRa SX1276 yang dihubungkan dengan mikrokontroller ESP32. Modul LoRa yang digunakan memiliki *carrier frequency* 915MHz.

Pengujian terhadap kemampuan modul LoRa yang digunakan dilakukan dengan mengukur nilai RSSI dari sinyal yang diterima serta meninjau data yang terkirim.

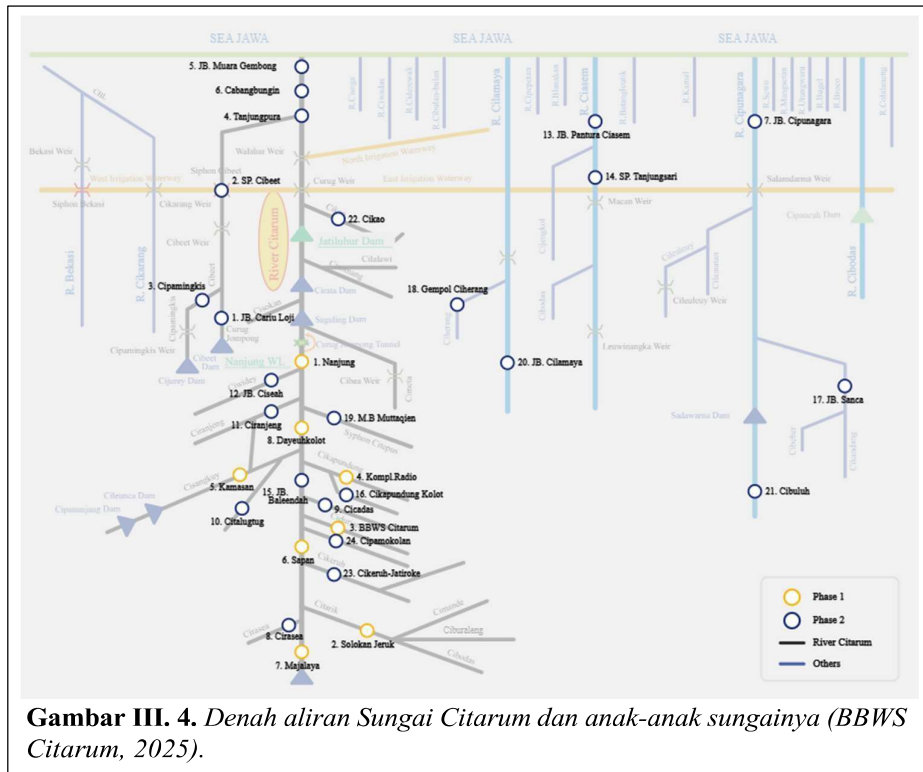
Pemantauan data yang diterima *receiver* LoRa dilakukan secara headless dengan menghubungkan *receiver* tersebut ke dalam sebuah jaringan MQTT. Pengujian dilakukan pada daerah padat bangunan di lingkungan Kampus Ganesha ITB, daerah tanah lapang di Lapangan Saraga ITB, dan daerah pepohonan di Hutan Kota Babakan Siliwangi.

III.3 Analisa Keberpengaruh data Curah Hujan terhadap Potensi Banjir

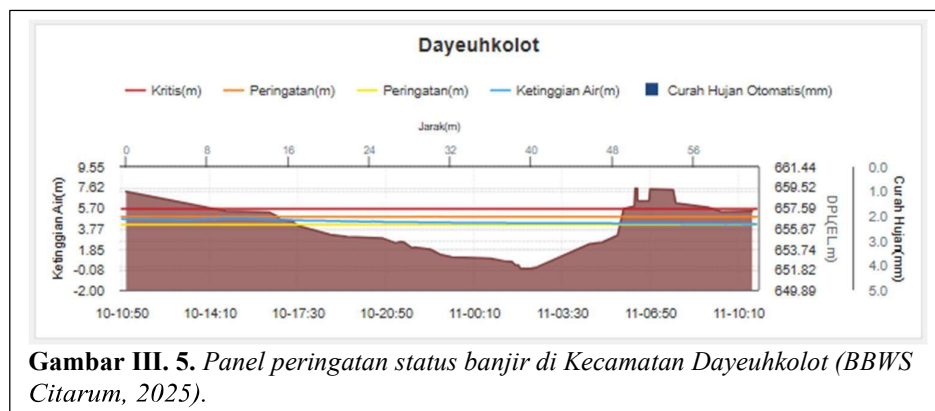
Untuk menentukan relasi antar pengukuran yang dilakukan oleh *sensor node*, data yang didapatkan dimodelkan dengan memanfaatkan library Scikit Learn sebagai model *linear regression* dan *decision tree*. Digunakan kedua model tersebut sebagai moda menganalisis keberpengaruh setiap pengukuran terhadap klasifikasi status kewaspadaan banjir yang ditetapkan oleh Balai Besar Wilayah Sungai (BBWS) Citarum berdasarkan ketinggian permukaan sungai (BBWS Citarum, 2025). Training dari model yang digunakan dilakukan dengan memanfaatkan data yang dikumpulkan oleh BBWS Citarum mengenai curah hujan pada sembilan titik PCH terhadap status kewaspadaan banjir. Berikut merupakan keterangan mengenai setiap pengukuran pada data yang dimodelkan.

Tabel III. 1. Keterangan Pengukuran Curah Hujan dan Ketinggian Permukaan Sungai yang dimodelkan berdasarkan data dari BBWS Citarum (BBWS Citarum, 2025).

Pengukuran	Nama	Koordinat
Pos Curah Hujan (PCH)	Sarijadi	-6.891864, 107.586283
	Dago Pakar	-6.861892, 107.624220
	Lembang	-6.826585, 107.617930
	Cimahi	-6.869677, 107.555057
	Tagog Apu	-6.811035, 107.473572
	Nanjung	-6.941818, 107.536373
	Paseh	-7.050545, 107.756667
	Cikapundung	-6.920210, 107.608704
	Dayeuhkolot	-6.984814, 107.616317
Pos Duga Air (PDA)	Dayeuhkolot	-6.984814, 107.616317



Setiap pengukuran curah hujan dan ketinggian permukaan sungai tersebut dilakukan dengan periode setiap sepuluh menit. Sehingga dapat dilakukan permodelan nilai setiap pengukuran curah hujan untuk menentukan nilai keberpengaruhan dari setiap PCH terhadap status kewaspadaan banjir berdasarkan pengukuran ketinggian permukaan sungai oleh PDA pada kecamatan Dayeuhkolot. Berikut merupakan kriteria status kewaspadaan banjir berdasarkan pengukuran-pengukuran tersebut.



Tabel III. 2. Keterangan status banjir berdasarkan ketinggian permukaan sungai pada PDA Dayeuhkolot (BBWS Citarum, 2025).

	Status banjir	Pelabelan pada model	Ketinggian permukaan sungai
PDA	Awas	3	>5.62
Dayeuhkolot	Siaga (2)	2	$4.88 < \dots < 5.62$
	Siaga (1)	1	$4.14 < \dots < 4.88$
	Aman	0	< 4.14

Dilakukan permodelan dengan model linear regression terlebih dahulu untuk menentukan hujan di daerah mana saja yang menyebabkan meningkatnya ketinggian permukaan sungai di Kecamatan Dayeuhkolot. Dikarenakan curah hujan dianggap selalu menambah ketinggian permukaan sungai, maka pada proses permodelan yang dilakukan koefisien-koefisien pada model persamaan linear dipaksa selalu bernilai positif. Sementara dikarenakan selalu ada air yang mengalir pada Sungai Citarum meskipun tidak terjadi hujan selama pengukuran yang dimodelkan, maka tidak ditetapkan nilai *intercept* dari model persamaan linear untuk tetap nol. Nilai-nilai koefisien tersebut dinormalisasi untuk menunjukkan keberpengaruhannya dari hujan di berbagai daerah di Bandung terhadap status kewaspadaan banjir di Kecamatan Dayeuhkolot sebagai nilai *feature importance*.

Setelah dilakukan permodelan dengan algoritma *linear regression*, dilakukan permodelan dengan algoritma *decision tree* pengukuran-pengukuran curah hujan yang menyebabkan meningkatnya ketinggian permukaan Sungai Citarum di Kecamatan Dayeuhkolot. Dari hasil permodelan tersebut, didapatkan hasil pengkategorian status kewaspadaan banjir yang juga dapat diimplementasikan sebagai peringatan dini banjir di Kecamatan Dayeuhkolot. Selain itu, diolah nilai *feature importance* dari Decision Tree yang dihasilkan sebagaimana dijelaskan pada persamaan 3 dan 4.

BAB IV

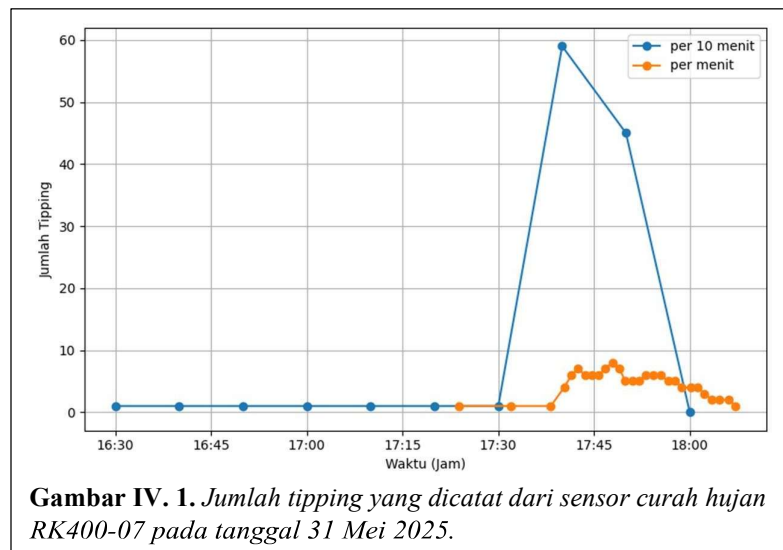
HASIL DAN ANALISIS

IV.1 Hasil Pengujian *Sensor node*

Berdasarkan pengujian awal terhadap kemampuan beberapa sub-sistem pada *sensor node* yang telah dilakukan, berikut merupakan hasil sementara yang didapatkan.

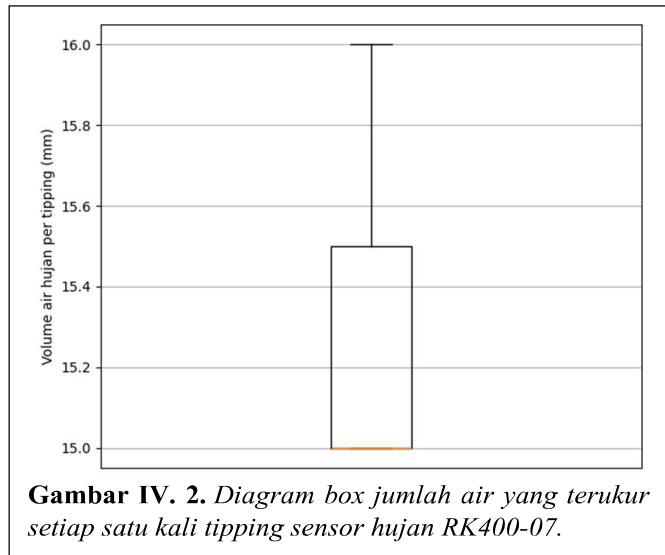
IV.1.1 Pengujian dan Kalibrasi *Rain gauge* RK400-07

Dilakukan pengujian *rain gauge* RK400-07 dengan membandingkan hasil pengukuran dari weather station di dekat tempat dilakukannya pengukuran curah hujan dengan *rain gauge* tersebut. Berikut merupakan grafik yang menunjukkan hasil pengukuran terhadap data dari weather station tersebut.



Grafik diatas menunjukkan sensor curah hujan dapat mencatat *tipping* akibat pengumpulan curah hujan dengan baik. Grafik berwarna biru pada gambar diatas menunjukkan jumlah *tipping* yang tercatat setiap sepuluh menit, yang merupakan hasil pengolahan data dari jumlah *tipping* total bila mengacu pada program pada sistem pengujian sensor curah hujan yang *flowchart*-nya ditunjukkan pada Gambar III.3b. Sementara grafik berwarna jingga menunjukkan jumlah curah hujan yang terukur selama satu menit setelah curah hujan pertama dicatat oleh program interrupt. Dapat diperhatikan bahwa sistem sensor curah hujan pada waktu

pengujian dilakukan mencatat maksimum delapan kali *tipping* dalam satu menit dan hampir enam puluh kali *tipping* selama periode antara jam 17.40 WIB sampai jam 17.49 WIB tanggal 31 Mei 2025. Untuk menentukan jumlah curah hujan yang terukur menurut *tipping* sensor curah hujan tersebut, kembali dirujuk besar curah hujan pada satu kali *tipping* sensor tersebut sebagaimana diberikan Rikasensor dan juga dilakukan kalibrasi sensor curah hujan sebagaimana berikut.

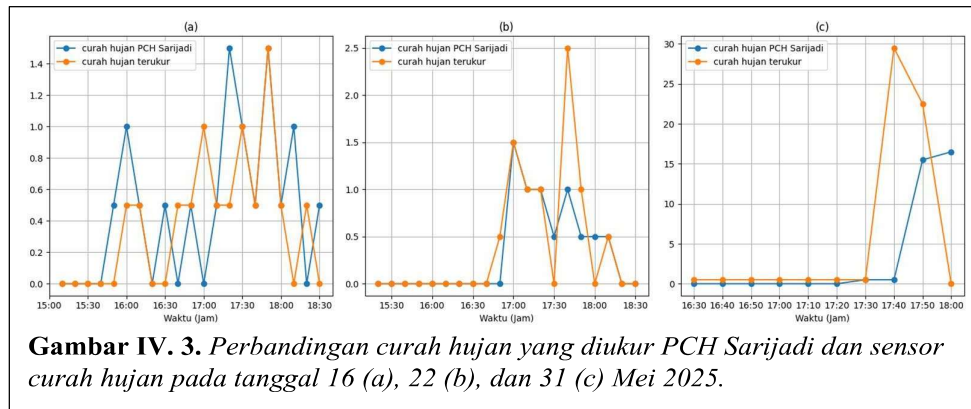


Gambar IV. 2. Diagram box jumlah air yang terukur setiap satu kali *tipping* sensor hujan RK400-07.

Dari hasil pengukuran volume air pada satu kali *tipping* sensor curah hujan RK400-07, didapatkan bahwa rata-rata 15.28 ml air menyebabkan satu kali *tipping* sensor tersebut, dengan Gambar IV.2 – yang menunjukkan diagram box dari pengukuran berulang volume air yang menyebabkan satu kali *tipping* – menunjukkan sensor lebih cenderung sudah melakukan satu kali *tipping* setelah diberikan air sejumlah 15 ml. Bilamana mengacu pada diameter sensor curah hujan sebesar 200 mm sebagaimana ditunjukkan pada Gambar II.1, maka satu kali *tipping* sensor curah hujan setara dengan curah hujan 0.49 mm, yang dapat dibulatkan menjadi 0.5 mm sebagaimana pada datasheet yang diberikan produsen dari sensor tersebut (RIKASENSOR, 2015).

Sehingga pada tanggal 31 Mei 2025, sensor curah hujan RK400-07 mengukur curah hujan maksimum 4 mm dalam satu menit, yang menurut keterangan yang diberikan produsen sensor tersebut, masih belum melebihi batas kemampuan sensor tersebut mencatat curah hujan dalam satu menit sebesar 8 mm (RIKASENSOR, 2015). Juga menurut produsen sensor tersebut, jumlah curah hujan yang terukur selama ~30 menit pengujian sensor – yaitu sebesar 63 mm – dapat dikategorikan sebagaimana

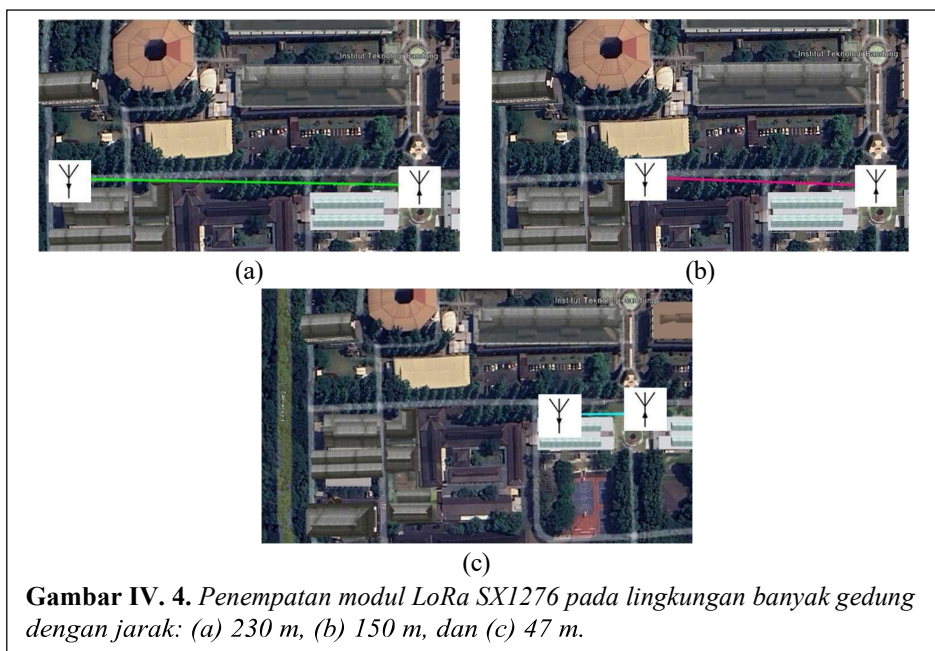
pada Tabel II.2 sebagai hujan *rainstorm* sebagaimana pada pengujian, atau *super-torrential* bila curah hujan yang terukur pada waktu pengujian dinormalisasi selama satu hari. Bila dibandingkan dengan pengukuran yang dilakukan PCH Sarijadi pada waktu yang sama, didapatkan hasil pengukuran berikut.



Dapat diperhatikan bahwa terdapat kemiripan antara pengukuran curah hujan dengan sensor yang diuji dan pengukuran yang dilakukan PCH Sarijadi. Perlu diperhatikan bahwa PCH Sarijadi berjarak 1040 m dari lokasi tempat pengujian sensor curah hujan, sehingga terdapat variasi spasial dari curah hujan yang terukur kedua sistem pengukuran tersebut. Namun grafik pada Gambar IV. 3 mengindikasikan bahwa sistem pengukuran curah hujan yang dirancang dan diuji pada penelitian ini berfungsi dengan baik dan dapat digunakan dalam sebuah sistem peringatan dini banjir.

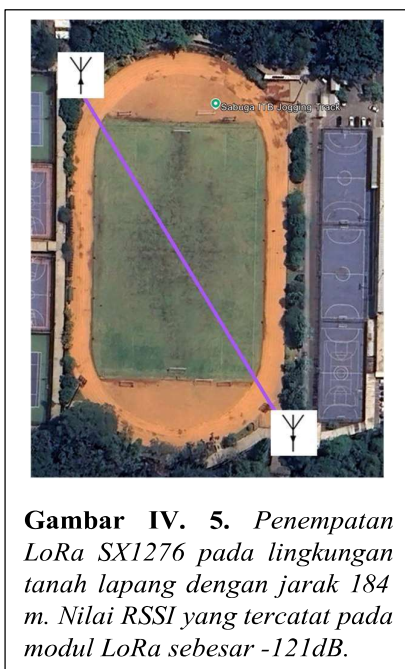
IV.1.2 Pengujian Kemampuan Modul LoRa SX1276

Dari pengujian modul LoRa SX1276 pada berbagai kondisi lingkungan, dihasilkan pengamatan jangkauan modul LoRa beserta nilai RSSI sebagai berikut.



Tabel IV. 1. Tabel nilai RSSI terhadap jarak antara dua modul LoRa SX1276 pada lingkungan banyak gedung.

Jarak (m)	Nilai RSSI (dB)
47	-112
150	-115
230	-121

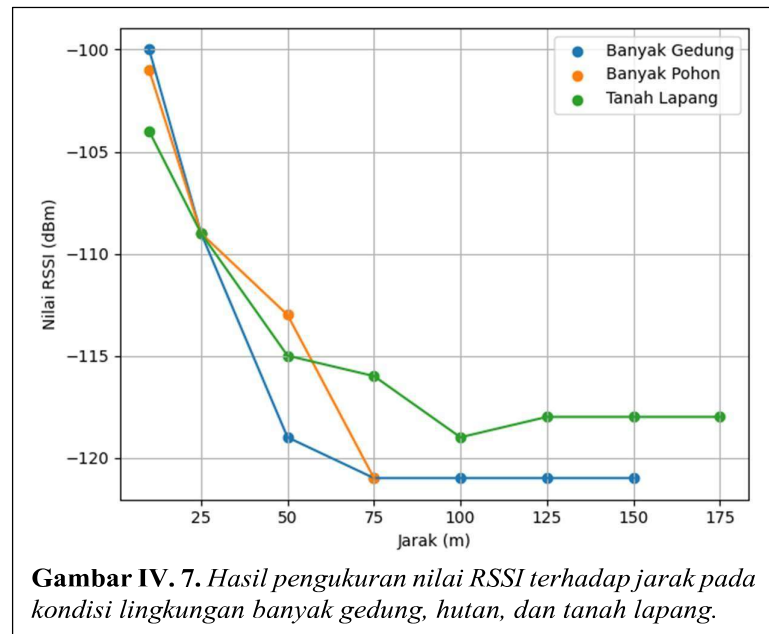




Tabel IV. 2. Tabel nilai RSSI terhadap jarak antara dua modul LoRa SX1276 pada lingkungan banyak pohon.

Jarak (m)	Nilai RSSI (dB)
18	-104
146	-121

Dari data tersebut, dapat di-plot nilai RSSI terhadap jarak pengamatan yang ditunjukkan pada grafik berikut.



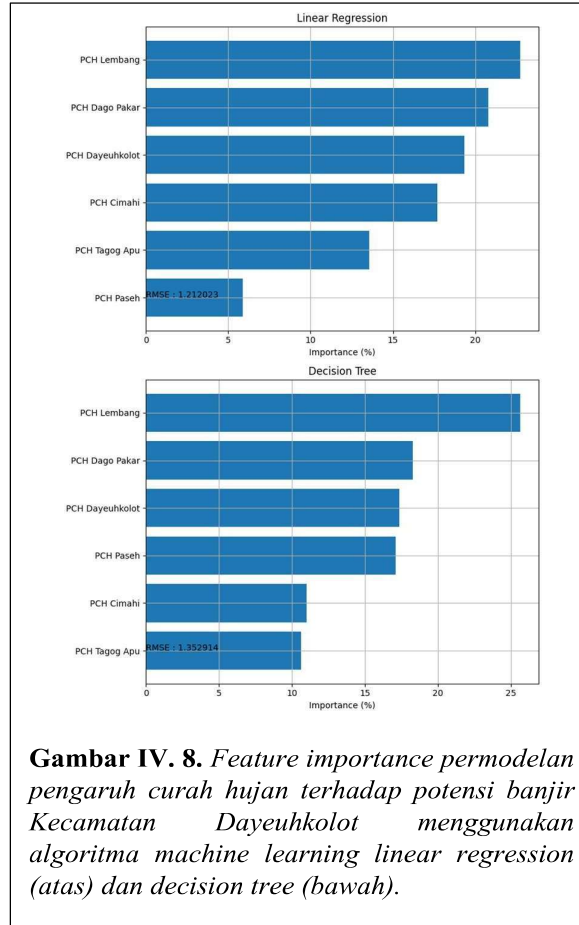
Dapat diperhatikan dari data-data yang telah dikumpulkan mengenai kemampuan modul LoRa SX1276 pada lingkungan banyak gedung, hutan, dan tanah lapang,

modul LoRa tersebut memiliki kemampuan yang relatif buruk berdasarkan keterangan nilai RSSI yang ditunjukkan pada Tabel II.1. Grafik diatas menunjukkan bahwa modul LoRa yang digunakan memiliki kemampuan paling baik pada kondisi lingkungan tanah lapang. Namun sinyal kualitas sinyal yang diterima LoRa masih relatif buruk. Hal tersebut menandakan bahwa terdapat pengaruh interferensi lingkungan seperti efek pemantulan dan hamburan terhadap gelombang pada transmisi dengan memanfaatkan LoRa (Sallyna et al., 2020), namun dapat tidak menjadi faktor utama yang menyebabkan buruknya kualitas sinyal LoRa yang diterima.

Hasil yang didapatkan juga menunjukkan kualitas sinyal golongan baik diterima hanya pada jarak sepuluh meter – yang dalam implementasi modul LoRa untuk mentransmisikan data dari sensor node menuju *monitoring station* di seberang sungai masih mungkin dilakukan dengan baik. Pada kondisi lingkungan yang representatif terhadap lokasi pengoperasian sensor node – yaitu pada lingkungan banyak gedung – transmisi LoRa dengan menggunakan modul tersebut masih dapat dilakukan hingga *monitoring station* berarak maksimum 150 meter dari sensor node.

IV.2 Hasil Analisa Keberpengaruh

Dilakukan permodelan dengan memanfaatkan algoritma *machine learning linear regression* dan *decision tree* untuk mendapatkan *feature importance* dari masing masing PCH terhadap potensi banjir pada PDA Dayeuhkolot. Sehingga didapatkan *feature importance* sebagaimana berikut:



Dapat diperhatikan pada Gambar IV.7 bahwa hujan di Kecamatan Dayeuhkolot sebagaimana tercatat oleh PCH di tempat tersebut mempengaruhi ketinggian air sungai – yang meningkatkan status kewaspadaan banjir. Namun hujan di Kecamatan Dayeuhkolot bukanlah faktor utama yang menimbulkan banjir di tempat tersebut. Hal tersebut dapat disebabkan oleh hujan yang turun di Kecamatan Dayeuhkolot segera dibawa ke arah barat laut oleh Sungai Citarum bila merujuk kembali ke skema aliran sungai Citarum sebagaimana ditunjukkan pada Gambar III. 4. Potensi banjir pada Kecamatan Dayeuhkolot utamanya dipengaruhi oleh hujan di Lembang dan Dago sebagaimana terukur curah hujan oleh PCH di daerah-daerah tersebut, yang dapat disebabkan oleh aliran sungai Cikapundung bertemu dengan Sungai Citarum sebelum melalui Kecamatan Dayeuhkolot.

Juga dapat diperhatikan bahwa nilai RMSE dari permodelan dengan menggunakan algoritma *linear regression* lebih kecil dibandingkan dengan nilai RMSE model dengan algoritma *decision tree*. Namun perlu diperhatikan bahwa algoritma *linear regression* menghasilkan prediksi secara kontinu, sehingga nilai error dari setiap

prediksi dapat lebih kecil dibandingkan dengan hasil prediksi dengan algoritma *decision tree* – yang menghasilkan prediksi berupa kategori yang bersifat diskrit. Dalam implementasi permodelan tersebut sebagai peringatan dini banjir, digunakan model dari permodelan dengan algoritma *decision tree*. Namun untuk menjelaskan fenomena banjir di Kecamatan Dayeuhkolot dan pengaruh hujan di berbagai daerah di Bandung terhadap potensi banjir, digunakan model dari permodelan dengan algoritma *linear regression*.

Persamaan linear yang didapatkan dari permodelan dengan algoritma *linear regression* – sebagaimana dicontohkan pada Persamaan 1 – dapat diinterpretasikan sebagai kombinasi curah hujan dalam besaran tinggi dalam mm yang menghasilkan suatu kategori yang terkait dengan ketinggian sungai yang terukur PDA Dayeuhkolot. Nilai *intercept* yang didapatkan merepresentasikan ketinggian permukaan Sungai Citarum bila tidak ditambah dengan air hujan, sementara nilai *slope* merupakan besaran yang menormalisasi besaran tinggi curah hujan sehingga setara dengan kategori yang tidak memiliki besaran fisis. Dapat diperhatikan pada grafik diatas bahwa tidak semua pengukuran curah hujan dimodelkan dengan algoritma *decision tree* dan *linear regression* untuk menentukan keberpengaruhannya. Pengukuran curah hujan di daerah Sarijadi, Nanjung, dan Cikapundung tidak dimodelkan dikarenakan ditemukan pada permodelan dengan algoritma *linear regression* bahwa koefisien pada persamaan linear yang diatribusikan dengan pengukuran curah hujan pada daerah tersebut bernilai nol. Hal tersebut menunjukkan bahwa curah hujan pada daerah-daerah tersebut tidak mempengaruhi peningkatan ketinggian permukaan Sungai Citarum di Kecamatan Dayeuhkolot. Permodelan dengan algoritma *decision tree* tidak mengenali hal tersebut dikarenakan proses pembelajaran dengan algoritma *machine learning* tersebut hanya dapat mengenali pola nilai-nilai curah hujan tertentu terhadap kategori status kewaspadaan banjir yang sama atau mayoritas pada *subset* data tersebut. Sehingga dari pemahaman mengenai fenomena banjir di Kecamatan Dayeuhkolot yang didapatkan dari analisa model dengan permodelan algoritma *linear regression* dilakukan permodelan dengan algoritma *machine learning decision tree* dengan tanpa pengukuran curah hujan pada daerah Sarijadi, Nanjung, dan Cikapundung – yang menghasilkan nilai *feature importance* yang mirip mengindikasikan bahwa dari dua metode permodelan *machine learning* dihasilkan model yang serupa.

BAB V

SIMPULAN DAN SARAN

V.1 Simpulan

Dari pengujian terhadap sistem pengukuran curah hujan dengan *sensor node* yang dilakukan beserta *sensor node* sebagai bagian dari sistem peringatan dini banjir yang dibangun, dapat diambil beberapa kesimpulan berikut.

Sensor curah hujan RK400-07 yang digunakan dalam pengukuran curah hujan dapat berfungsi dengan baik. Satu kali *tipping* dari sensor *tipping* bucket tersebut berkorespondensi dengan 15.875mm air hujan atau setara dengan curah hujan 0.5mm. Satu kali *tipping* tersebut menghasilkan satu sinyal pulsa bila sensor dirangkai sebagaimana pada Gambar II.1b dan diproses dengan menggunakan mikrokontroller ESP32 untuk dicatat pada *data logger*. Hasil pencatatan tersebut ditunjukkan pada Gambar IV.1 untuk jumlah *tipping* per menit dan per sepuluh menit pengukuran hujan pada tanggal 31 Mei 2025 – yang menunjukkan sensor tersebut dapat mengukur curah hujan *rainstorm* menurut skala pada Tabel II.2.

Modul LoRa SX1276 yang digunakan untuk mentransmisikan data dari *sensor node* ke *monitoring station* diuji pada kondisi lingkungan banyak gedung, banyak pohon, dan tanah lapang. Melalui pengujian tersebut, ditemukan bahwa lingkungan mempengaruhi kualitas sinyal LoRa, dengan kualitas sinyal terbaik terdapat pada transmisi pada lingkungan tanah lapang. Pada lingkungan yang merepresentasikan pengoperasian *sensor node* pada Sungai Citarum, ditemukan bahwa modul LoRa tersebut dapat mentransmisikan data sejauh ~140 m.

Dilakukan analisa keberpengaruhan curah hujan pada berbagai daerah terhadap potensi banjir pada Kecamatan Dayeuhkolot dan Kecamatan Baleendah sebagaimana dilakukan pengukuran oleh PCA dan PDA yang tertera pada Tabel III. 2. Ditemukan bahwa status kewaspadaan banjir pada Kecamatan Dayeuhkolot paling dipengaruhi oleh hujan di daerah Dago dan Lembang sebelum dipengaruhi oleh hujan di daerah tersebut – yang dapat disebabkan oleh aliran Sungai Cikapundung yang bertemu dengan Sungai Citarum sebelum melalui Kecamatan Dayeuhkolot. Model yang dihasilkan dari permodelan dengan algoritma *linear regression* dan *decision tree* menghasilkan kesimpulan keberpengaruhan yang sama, dengan model *decision tree* digunakan sebagai peringatan dini banjir di

Kecamatan Dayeuhkolot dan model *linear regression* digunakan untuk memahami pengaruh hujan di Bandung terhadap potensi banjir di Kecamatan Dayeuhkolot.

V.2 Saran

Dari penelitian yang telah dilakukan, penulis menyarankan beberapa hal berikut untuk dikaji lebih lanjut.

Untuk sistem peringatan dini banjir yang dibangun, dapat dikaji lebih lanjut kemampuan baik keseluruhan *sensor node* atau sistem-sistem selain sistem pengukuran curah hujan dalam *sensor node* tersebut dalam kondisi hujan, bagaimana kualitas transmisi LoRa dapat dipengaruhi oleh hujan dan bagaimana ketahanan sistem penyuplai daya bilamana terjadi hujan yang berkeberlanjutan. Juga sistem komunikasi dan pengelolaan data pada sistem peringatan dini banjir tersebut, dapat dikaji mengenai periode query yang optimal untuk meminimalisasi penggunaan daya tanpa mengurangi nilai data dalam permodelan dengan algoritma *machine learning* yang digunakan.

Untuk permodelan dengan memanfaatkan algoritma *machine learning* yang dilakukan, dapat dikaji pengaruh pola cuaca di Bandung dalam satu tahun bila dilakukan permodelan yang sama. Melalui kajian lebih lanjut terhadap pola cuaca yang menandakan hujan juga dapat diimplementasikan pengukuran suhu dan kelembaban yang dilakukan sensor DHT11 pada *sensor node* untuk melakukan permodelan yang lebih lengkap mengenai potensi banjir suatu daerah yang dikaji pada DAS Citarum. Juga data yang sama juga dapat dimodelkan sebagai time-series menggunakan ARIMA atau permodelan data *time series* lainnya untuk mengembangkan sistem yang dibangun dari sistem peringatan dini banjir menjadi sistem prediksi banjir.

Pada penelitian ini, dilakukan analisa keberpengaruhan dan ditemukan secara empiris nilai *feature importance* dari curah hujan dari setiap daerah yang dimodelkan. Namun juga dapat dikaji makna fisis dari besaran tersebut sebagai faktor atau kombinasi dari sistem-sistem yang mempengaruhi drainase dan aliran air hujan dari masing-masing daerah tersebut menuju lokasi kajian potensi banjir seperti lebar sungai, kelandaian tempat terukurnya hujan, serta pengaruh sampah dan aktivitas masyarakat.

DAFTAR PUSTAKA

- A'Ziziyyah, A. A., Nugroho, I. I., Sabillillah, R., Aji, B. A. S., & Amiroh, K. (2022). Perbandingan Sistem Deteksi Banjir Menggunakan Algoritma Naive Bayes Dan K-NN Berbasis IOT. *IJCIT (Indonesian Journal on Computer and Information Technology)*, 7(1).
- Altmann, A., Tološi, L., Sander, O., & Lengauer, T. (2010). Permutation importance: a corrected *feature importance* measure. *Bioinformatics*, 26(10), 1340-1347.
- BBWS Citarum, B. B. W. S. C. (2025). *Citarum River Flood Forecasting and Warning System*
- BNPB, B. N. P. B. (2024). *Jumlah Kejadian Bencana Alam Menurut Provinsi, 2023*.
- Bor, M. C., Vidler, J., & Roedig, U. (2016, 2016). LoRa for the Internet of Things. International Conference on Embedded Wireless Systems and Networks (EWSN), Graz, Austria.
- Chaudhary, A., Kolhe, S., & Kamal, R. (2016). An improved random forest classifier for multi-class classification. *Information Processing in Agriculture*, 3(4), 215-222. <https://doi.org/https://doi.org/10.1016/j.inpa.2016.08.002>
- Crawford, F. S. (1968). *Waves, Berkeley Physics Course Volume 3*. McGraw-Hill.
- Douben, K. J. (2006). Characteristics of river floods and flooding: a global overview, 1985 — 2003. *Irrigation and Drainage: The journal of the International Commission on Irrigation and Drainage*, 55(S1), S9-S21.
- Gkotsiopoulou, P., Zorbas, D., & Douligieris, C. (2021). Performance determinants in LoRa networks: A literature review. *IEEE Communications Surveys & Tutorials*, 23(3), 1721-1758.
- Hadi, M. I., Yakub, F., Fakhrurrazi, A., Hui, C. X., Najiha, A., Fakharulrazi, N. A.,...Azizan, A. (2020, 2020). Designing early warning flood detection and monitoring system via IoT. The 7th AUN/SEED-Net Regional Conference on Natural Disaster,
- Hagino, T. (2021). *Practical Node-RED Programming: Learn powerful visual programming techniques and best practices for the web and IoT*. Packt Publishing Ltd.
- Hercog, D., Lerher, T., Truntič, M., & Težak, O. (2023). Design and implementation of ESP32-based IoT devices. *Sensors*, 23(15), 6739.
- Hillar, G. C. (2017). *MQTT Essentials-A lightweight IoT protocol*. Packt Publishing Ltd.
- Jayantara, I. G. N. Y. (2020). Implementasi QGIS untuk mengestimasi kerugian ekonomi akibat banjir di Kabupaten Bandung. *Jurnal Pendidikan Teknologi dan Kejuruan*, 17(2), 231-242.
- Peraturan Direktur Jenderal Sumber Daya dan Perangkat Pos dan Informatika Nomor 3 Tahun 2019 tentang Persyaratan Teknis Alat dan/atau Perangkat Telekomunikasi Low Powe Wide Area, (2019).

https://sdppi.kominfo.go.id/downloads/42/20210510124739-perdirjen_2019-03_LPWA.pdf

- Kreibich, H., & Dimitrova, B. (2010). Assessment of damages caused by different flood types. *WIT Transactions on Ecology and the Environment*, 133, 3-11.
- Krzyszczanovskaya, V. V., Shirshov, G. S., Melnikova, N. B., Belleman, R. G., Rusadi, F. I., Broekhuijsen, B. J.,...Meijer, R. J. (2011). Flood early warning system: design, implementation and computational modules. *Procedia Computer Science*, 4, 106-115. <https://doi.org/10.1016/j.procs.2011.04.012>
- Kurniawan, A. (2019). *Internet of Things Projects with ESP32: Build exciting and powerful IoT projects using the all-new Espressif ESP32*. Packt Publishing Ltd.
- Li, S., Xu, L. D., & Zhao, S. (2015). The internet of things: a survey. *Information systems frontiers*, 17, 243-259.
- Liu, C. (1997). Multimedia over ip: Rsvp, rtp, rtcp, rtsp. *Handbook of emerging communications technologies: the next decade*, 29-46.
- Maulud, D., & Abdulazeez, A. M. (2020). A review on linear regression comprehensive in machine learning. *Journal of applied science and technology trends*, 1(2), 140-147.
- Muzakky, A., Nurhadi, A., Nurdiansyah, A., & Wicaksana, G. (2019, 2018). Perancangan sistem deteksi banjir berbasis IoT. Conference on Innovation and Application of Science and Technology 2019,
- Naqvi, S. N. Z., Yfantidou, S., & Zimányi, E. (2017). Time series databases and influxdb. *Studienarbeit, Université Libre de Bruxelles*, 12, 1-44.
- Nashiruddin, M. I., & Hidayati, A. (2019, 2019). Coverage and capacity analysis of LoRa WAN deployment for massive IoT in urban and suburban scenario. 2019 5th International Conference on Science and Technology (ICST), Yogyakarta, Indonesia.
- Perera, D., Seidou, O., Agnihotri, J., Rasmy, M., Smakhtin, V., Coulibaly, P., & Mehmood, H. (2019). Flood early warning systems: a review of benefits, challenges and prospects. *UNU-INWEH, Hamilton*.
- Prakosa, S. W., Faisal, M., Adhitya, Y., Leu, J.-S., Köppen, M., & Avian, C. (2021). Design and implementation of LoRa based IoT scheme for Indonesian rural area. *Electronics*, 10(1), 77.
- RIKASENSOR. (2015). RK400-07 *Tipping* Bucket Rainfall Sensor. In L. Hunan Rika Electronic Tech Co. (Ed.). Changsha City, Hunan Province, China.
- Sallyna, D., Usman, U. K., & Murti, M. A. (2020). Perencanaan Jaringan Long Range (Lora) Pada Frekuensi 920 Mhz-923 Mhz Di Kota Bandung Long Range (Lora) Network Planning With Frequency 920 Mhz-923 Mhz In Bandung City. *e-Proceeding of Engineering*, 7(1), 933-940.
- Schulzrinne, H., Rao, A., & Lanphier, R. (1998). RFC2326: Real time streaming protocol (RTSP). In: RFC Editor.
- SEMTECH. (2020). SX1276/77/78/79 - 137MHz to 1020 MHz Low Power Long Range Transceiver. In (Rev. 7 ed., pp. 132). Camarillo, California: Semtech Corporation.

- Simanjuntak, Y. S. M., Suwarman, R., & Edi, R. (2023). ANALISIS KARAKTERISTIK CURAH HUJAN PENYEBAB BANJIR BERDURASI PANJANG (STUDI KASUS: BANJIR TAHUN 2019 DI BALEENDAH, JAWA BARAT). *JURNAL SUMBER DAYA AIR*, 19(1), 29-41.
- Sitorus, I. H. O., Bioresita, F., & Hayati, N. (2021). Analisa tingkat rawan banjir di daerah kabupaten bandung menggunakan metode pembobotan dan scoring. *Jurnal Teknik ITS*, 10(1), C14-C19.
- Sun, J. (2012). Pulse-width modulation. In *Dynamics and Control of Switched Electronic Systems: Advanced perspectives for modeling, simulation and control of power converters* (pp. 25-61). Springer.
- Szeliski, R. (2022). *Computer vision: algorithms and applications*. Springer Nature.
- Tadrist, N., Debauche, O., Mahmoudi, S., & Guttadauria, A. (2022). Towards Low-Cost IoT and LPWAN-Based flood forecast and monitoring system. *International journal of ubiquitous systems and pervasive networks*, 17(1).
- Terven, J., Córdova-Esparza, D.-M., & Romero-González, J.-A. (2023). A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas. *Machine learning and Knowledge Extraction*, 5(4), 1680-1716.
- Torres, W. P., Oppenheim, A. V., & Rosales, R. R. (2001). Generalized frequency modulation. *IEEE Transactions on Circuits and Systems I: Fundamental theory and applications*, 48(12), 1405-1412.
- Windiastrik, S. P., Ardhana, E. N., & Triono, J. (2019, 2019). Perancangan Sistem Pendeteksi Banjir Berbasis IoT (Internet of Thing). Seminar Nasional Teknologi Informasi 2019,

LAMPIRAN A

PEMROGRAMAN

A.1 Program Arduino Sensor Node

```
#include <TinyGPS++.h>
#include <SPI.h>
#include <LoRa.h>
#include <DHT.h>
#define rainsensor_pin 13
#define ss 5
#define rst 14
#define dio0 2
#define DHTPIN 12 //masukkan pin digital DHT11
#define DHTTYPE DHT11

TinyGPSPlus gps;
DHT dht(DHTPIN, DHTTYPE);

int rainsensor_minutely;
int rainsensor_count;
int rainsensor_status;
float sensortimer;
float sendtimer;
float temp;
float hum;

void rainsensor() {
    sensortimer = millis();
    rainsensor_status = 1;
}

void setup() {
    Serial.begin(115200);
    Serial2.begin(9600, SERIAL_8N1, 16, 17);
    LoRa.setPins(ss, rst, dio0);
    while (!LoRa.begin(915E6)) {
        delay(500);
    }
    LoRa.setSyncword(0xF3);
    sensortimer = millis();
    sendtimer = millis();
    rainsensor_minutely = 0;
    rainsensor_count = 0;
    rainsensor_status = 0;
    pinMode(rainsensor_pin, INPUT);
    attachInterrupt(digitalPinToInterrupt(rainsensor_pin), rainsensor, RISING);
}

void loop() {
    if (rainsensor_status == 1) {
        rainsensor_minutely += 1;
        rainsensor_count += 1;
        rainsensor_status == 2;
    }

    if ((millis() - sensortimer) >= 60000 && rainsensor_status == 2 && Serial2.available() > 0) {
```

```

        if (gps.encode(Serial2.read())) {
            if (gps.location.isValid()) {
                temp = dht.readTemperature();
                hum = dht.readHumidity();
                LoRa.beginPacket();
                LoRa.print(gps.location.lat(), 3);
                LoRa.print(",");
                LoRa.print(gps.location.lng(), 3);
                LoRa.print(",");
                LoRa.print(sensortimer, 3);
                LoRa.print(",");
                LoRa.print(rainsensor_minutely);
                LoRa.print(",");
                LoRa.print(rainsensor_count);
                LoRa.print(",");
                LoRa.print(temp);
                LoRa.print(",");
                LoRa.print(hum);
                LoRa.println();
                LoRa.endPacket();
                rainsensor_minutely = 0;
                rainsensor_status = 0;
                sendtimer = millis();
            }
        }
    } else if ((millis() - sendtimer) > 5000 && rainsensor_status == 2 && gps.charsProcessed()
    < 10) {
        LoRa.beginPacket();
        LoRa.println("No GPS data! Check wiring!");
        LoRa.endPacket();
    }
}
}

```

A.2 Program Arduino Monitoring Station

```

#include <WiFi.h>
#include <PubSubClient.h>
#include <SPI.h>
#include <LoRa.h>
#define ss 5
#define rst 14
#define dio0 2

const char *ssid = "ssid_wifi"; // ganti menjadi nama wifi anda
const char *password = "password_wifi"; // ganti menjadi password wifi anda
const char *mqtt_broker = "123.456.789.123"; // ganti menjadi IP server anda
const char *topic = "ta_sensornode";
const char *mqtt_username = "mqtt_user"; // ganti menjadi username server anda
const char *mqtt_password = "mqtt_password"; // ganti menjadi password server anda
const int mqtt_port = 1883;

WiFiClient espClient;
PubSubClient client(espClient);

void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
    }
    client.setServer(mqtt_broker, mqtt_port);
    client.setCallback(callback);
}

```

```

while(!client.connected()) {
  String client_id = "esp32-client-";
  client_id += String(WiFi.macAddress());
  if (client.connect(client_id.c_str(), mqtt_username, mqtt_password)) {
    Serial.print("Connected to MQTT!");
  } else {
    Serial.print("failed with state: ");
    Serial.print(client.state());
    Serial.println();
    delay(500);
  }
}
client.subscribe(topic);
LoRa.setPins(ss, rst, dio0);
while (!LoRa.begin(915E6)) {
  delay(500);
}
LoRa.setSyncWord(0xF3);
}

void callback(char *topic, byte *payload, unsigned int length) {
  Serial.println("Received message from topic!");
}

void loop() {
  int packetSize = LoRa.parsePacket();
  if(packetSize) {
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      client.publish(topic, LoRaData.c_str());
    }
  }
}

```

A.3 Program Arduino Pengujian Rainfall Sensor

```

#define rainsensor_pin 13

int rainsensor_minutely;
int rainsensor_count;
int rainsensor_status;
float sensortimer;
float sendtimer;

void rainsensor() {
  sensortimer = millis();
  rainsensor_status = 1;
}

void setup() {
  Serial.begin(115200);
  sensortimer = millis();
  sendtimer = millis();
  rainsensor_minutely = 0;
  rainsensor_count = 0;
  rainsensor_status = 0;
  pinMode(rainsensor_pin, INPUT);
  attachInterrupt(digitalPinToInterrupt(rainsensor_pin), rainsensor, RISING);
}

void loop() {
  if (rainsensor_status == 1) {

```

```

    rainsensor_minutely += 1;
    rainsensor_count += 1;
    rainsensor_status == 2;
}

if ((millis() - sensortimer) >= 60000 && rainsensor_status == 2) {
    Serial.print(sensortimer, 3);
    Serial.print(";");
    Serial.print(rainsensor_minutely);
    Serial.print(";");
    Serial.print(rainsensor_count);
    Serial.println();
    rainsensor_minutely = 0;
    rainsensor_status = 0;
    sendtimer = millis();
}
}
}

```

A.4 Program Python Pengujian Rainfall Sensor

```

# Import libraries
import pandas as pd
import numpy as np
import random as rd
import pprint as pp
import matplotlib.pyplot as plt

from sklearn.utils import resample
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
from sklearn.metrics import root_mean_squared_error

# Import file
DF = pd.read_excel(r'C:/Users/Pongo/Downloads/Data Tensorflow TA.xlsx')

# Management functions
def data_upsample(ORIGINAL_FEATURES, ORIGINAL_LABEL):
    ORIGINAL_DATA = np.column_stack([ORIGINAL_FEATURES, ORIGINAL_LABEL])
    while True:
        POP = [], []
        POP[0], POP[1] = np.unique(ORIGINAL_DATA[:, -1], return_counts = True)
        pop_maj = POP[0][np.argmax(POP[1])]
        pop_min = POP[0][np.argmin(POP[1])]
        if len(ORIGINAL_DATA[ORIGINAL_DATA[:, -1] == pop_min])*10 >=
len(ORIGINAL_DATA[ORIGINAL_DATA[:, -1] == pop_maj]):
            return ORIGINAL_DATA

        DATA_ELSE = ORIGINAL_DATA[ORIGINAL_DATA[:, -1] != pop_min]
        DATA_MIN = ORIGINAL_DATA[ORIGINAL_DATA[:, -1] == pop_min]
        FEATURES_UPSAMPLE, LABEL_UPSAMPLE = resample(DATA_MIN[:, :-1], DATA_MIN[:, -1],
replace = True, n_samples = len(ORIGINAL_DATA[ORIGINAL_DATA[:, -1] == pop_maj]), random_state
= 0)

        NEW_FEATURES = np.concatenate((FEATURES_UPSAMPLE, DATA_ELSE[:, :-1]))
        NEW_LABEL = np.concatenate((LABEL_UPSAMPLE, DATA_ELSE[:, -1]))
        ORIGINAL_DATA = np.column_stack([NEW_FEATURES, NEW_LABEL])

def look_ahead(INPUT_DATA, n_lookahead):
    INPUT_DATA_FEATURES = INPUT_DATA[:-n_lookahead, :-1]
    INPUT_DATA_LABEL = INPUT_DATA[n_lookahead:, -1]
    return np.column_stack([INPUT_DATA_FEATURES, INPUT_DATA_LABEL])

```

```

def list_sort(LIST):
    SORTED_LIST = [[], []]
    while len(LIST[1]) > 0:
        SORTED_LIST[0].append(LIST[0][LIST[1].index(min(LIST[1]))])
        SORTED_LIST[1].append(LIST[1][LIST[1].index(min(LIST[1]))])
        LIST[0].pop(LIST[1].index(min(LIST[1])))
        LIST[1].pop(LIST[1].index(min(LIST[1])))
    return SORTED_LIST

def class_accuracy(RESULT, ACTUAL, title = "Accuracy :"):
    ACCURACY_TABLE = np.zeros((len(np.unique(RESULT)), len(np.unique(ACTUAL))))
    for j in range(len(RESULT)):
        ACCURACY_TABLE[int(RESULT[j]), int(ACTUAL[j])] += 1
    ACCURACY_TABLE = ACCURACY_TABLE/len(RESULT)
    ACCURACY_TABLE = pd.DataFrame(data = ACCURACY_TABLE, index = ["Result %i" % int(n) for n
in np.unique(RESULT)], columns = ["Actual %i" % int(n) for n in np.unique(ACTUAL)])
    print(title)
    print(ACCURACY_TABLE)
    print("RMSE : %f" % root_mean_squared_error(ACTUAL, RESULT))
    print()

# Linear regression modelling functions
def linreg_model(FEATURES_MODEL, INTERCEPT_MODEL, slope_model):
    LABEL_MODEL = np.zeros(FEATURES_MODEL.shape[0])
    for i in range(FEATURES_MODEL.shape[0]):
        LABEL_MODEL[i] = np.dot(INTERCEPT_MODEL, FEATURES_MODEL[i]) + slope_model
    return LABEL_MODEL

def linreg_mse(FEATURES_MSE, CURRENT_LABEL, INTERCEPT_MSE, slope_mse):
    NEW_LABEL = linreg_model(FEATURES_MSE, INTERCEPT_MSE, slope_mse)
    return np.sum((NEW_LABEL - CURRENT_LABEL)**2)/(2*INTERCEPT_MSE.shape[0])

def linreg_iter(FEATURES_ITER, LABEL_ITER, INTERCEPT_ITER, slope_iter, slope_calc_status):
    slope_delta = 0
    INTERCEPT_DELTA = np.zeros(FEATURES_ITER.shape[1])
    for i in range(FEATURES_ITER.shape[0]):
        error = np.dot(INTERCEPT_ITER, FEATURES_ITER[i]) + slope_iter - LABEL_ITER[i]
        INTERCEPT_DELTA += error*FEATURES_ITER[i]
        if slope_calc_status:
            slope_delta += error
    INTERCEPT_DELTA /= FEATURES_ITER.shape[0]
    if slope_calc_status:
        slope_delta /= FEATURES_ITER.shape[0]
    return [slope_delta, INTERCEPT_DELTA]

def linreg_fit(FEATURES_FIT, LABEL_FIT, INTERCEPT = [], slope = 0, slope_calc = True, force_nn
= False, fitting_rate = 1E-1, epoch = 1E3, return_plots = False):
    if INTERCEPT == []:
        INTERCEPT = np.ones(shape = (FEATURES_FIT.shape[1]))
    if slope_calc == False and slope != 0:
        slope = 0
    for n in range(int(epoch)):
        [slope_fit, INTERCEPT_FIT] = linreg_iter(FEATURES_FIT, LABEL_FIT, INTERCEPT, slope,
slope_calc)
        slope = slope - fitting_rate*slope_fit
        INTERCEPT = INTERCEPT - fitting_rate*INTERCEPT_FIT
        if force_nn:
            INTERCEPT[INTERCEPT < 0] = 0
    if return_plots:
        FEATURE_NAME = DF.iloc[:, :-1].columns.to_list()
        fit_fig = plt.figure(figsize = (4, FEATURES_FIT.shape[1]*3))
        fit_ax = fit_fig.subplots(FEATURES_FIT.shape[1], 1)
        for j in range(FEATURES_FIT.shape[1]):

```

```

fit_ax[j].scatter(FEATURES_FIT[:, j], LABEL_FIT)
fit_ax[j].plot(FEATURES_FIT[:, j], FEATURES_FIT[:, j]*INTERCEPT[j] + slope)
fit_ax[j].set_xlabel(FEATURE_NAME[j])
fit_ax[j].set_ylabel("Label")
fit_ax[j].legend(["Training values", "Model"])
fit_ax[j].grid()
plt.tight_layout()
plt.show()
plt.close(fit_fig)
return [INTERCEPT, slope]

# Random forest modelling functions
def dtree_purity(LABEL_PURITY):
    UNIQUE_LABEL = np.unique(LABEL_PURITY)
    if len(UNIQUE_LABEL) == 1:
        purity_status = True
    else:
        purity_status = False
    return purity_status

def dtree_class(LABEL_CLASS):
    UNIQUE_CLASS = [[], []]
    UNIQUE_CLASS[0], UNIQUE_CLASS[1] = np.unique(LABEL_CLASS, return_counts = True)
    mode_index = UNIQUE_CLASS[1].argmax()
    return UNIQUE_CLASS[0][mode_index]

def dtree_pot(FEATURES_POT, subspace_pot):
    SPLIT_POTENTIAL = {}
    INDEX_POT = [i for i in range(FEATURES_POT.shape[1])]
    if subspace_pot and subspace_pot <= len(DTREE_INFO['IMPORTANCES']):
        INDEX_POT = rd.sample(population = INDEX_POT, k = subspace_pot)
    for i in INDEX_POT:
        FEATURE_VALUES = FEATURES_POT[:, i]
        SPLIT_POTENTIAL[i] = np.unique(FEATURE_VALUES)
    return SPLIT_POTENTIAL

def dtree_split(FEATURES_SPLIT, LABEL_SPLIT, split_index, split_value):
    DATA_SPLIT = np.column_stack([FEATURES_SPLIT, LABEL_SPLIT])
    SPLIT_PARAMS = DATA_SPLIT[:, split_index]
    DATA_LOWER = DATA_SPLIT[SPLIT_PARAMS <= split_value]
    DATA_HIGHER = DATA_SPLIT[SPLIT_PARAMS > split_value]
    return [DATA_LOWER, DATA_HIGHER]

def dtree_entropy(LABEL_ENTROPY):
    UNIQUE_ENTROPY = [[], []]
    UNIQUE_ENTROPY[0], UNIQUE_ENTROPY[1] = np.unique(LABEL_ENTROPY, return_counts = True)
    PROB = UNIQUE_ENTROPY[1]/UNIQUE_ENTROPY[1].sum()
    return sum(PROB*(-np.log2(PROB)))

def dtree_gini(LABEL_ENTROPY):
    UNIQUE_ENTROPY = [[], []]
    UNIQUE_ENTROPY[0], UNIQUE_ENTROPY[1] = np.unique(LABEL_ENTROPY, return_counts = True)
    PROB = UNIQUE_ENTROPY[1]/UNIQUE_ENTROPY[1].sum()
    return 1 - (PROB**2).sum()

def dtree_overall(LABEL_LOWER, LABEL_HIGHER):
    p_lower = len(LABEL_LOWER)/(len(LABEL_LOWER) + len(LABEL_HIGHER))
    p_higher = len(LABEL_HIGHER)/(len(LABEL_LOWER) + len(LABEL_HIGHER))
    return p_lower*dtree_gini(LABEL_LOWER) + p_higher*dtree_gini(LABEL_HIGHER)

def dtree_optimal(FEATURES_OPTIMAL, LABEL_OPTIMAL, SPLIT_VALUES):
    overall = 99
    for i in SPLIT_VALUES:

```

```

        for j in SPLIT_VALUES[i]:
            SPLITTED_LABEL = dtree_split(FEATURES_OPTIMAL, LABEL_OPTIMAL, i, j)
            new_overall = dtree_overall(SPLITTED_LABEL[0][:, -1], SPLITTED_LABEL[1][:, -1])
            if new_overall <= overall:
                overall = new_overall
            OPTIMAL_SPLIT = [i, j]
        return OPTIMAL_SPLIT

def dtree_fit(FEATURES_FIT, LABEL_FIT, branch_limit = 1E5, min_samples = 2, n_branch = 0,
n_subspace = None):
    if n_branch == 0:
        global DTREE_INFO
        DTREE_INFO = {}
        DTREE_INFO['n_data'] = len(LABEL_FIT)
        DTREE_INFO['IMPORTANCES'] = np.zeros(FEATURES_FIT.shape[1])
        DTREE_INFO['FEATURE_FREQUENCY'] = np.zeros(FEATURES_FIT.shape[1])
    if dtree_purity(LABEL_FIT):
        model_class = dtree_class(LABEL_FIT)
        return [model_class, 'reached_purity']
    elif len(LABEL_FIT) < min_samples:
        model_class = dtree_class(LABEL_FIT)
        return [model_class, 'reached_min_samples']
    elif n_branch >= branch_limit:
        model_class = dtree_class(LABEL_FIT)
        return [model_class, 'reached_branch_limit']
    else:
        n_branch += 1
        MODEL_POT = dtree_pot(FEATURES_FIT, n_subspace)
        OPTIMAL_MODEL = dtree_optimal(FEATURES_FIT, LABEL_FIT, MODEL_POT)
        SPLITTED_DATA = dtree_split(FEATURES_FIT, LABEL_FIT, OPTIMAL_MODEL[0],
OPTIMAL_MODEL[1])
        if len(SPLITTED_DATA[0]) == 0 or len(SPLITTED_DATA[1]) == 0:
            model_class = dtree_class(LABEL_FIT)
            return [model_class, 'didnt_find_split']
        branch = "{} <= {}".format(OPTIMAL_MODEL[0], OPTIMAL_MODEL[1])
        SUB_TREE = {branch: []}
        YES_BRANCH = dtree_fit(SPLITTED_DATA[0][:, :-1], SPLITTED_DATA[0][:, -1],
branch_limit, min_samples, n_branch, n_subspace)
        NO_BRANCH = dtree_fit(SPLITTED_DATA[1][:, :-1], SPLITTED_DATA[1][:, -1],
branch_limit, min_samples, n_branch, n_subspace)
        if YES_BRANCH == NO_BRANCH:
            SUB_TREE = YES_BRANCH
        else:
            SUB_TREE[branch].append(YES_BRANCH)
            SUB_TREE[branch].append(NO_BRANCH)
            gain = dtree_gini(LABEL_FIT)
            gain -= (len(SPLITTED_DATA[0][:, -1])*dtree_gini(SPLITTED_DATA[0][:, -1]) +
len(SPLITTED_DATA[1][:, -1])*dtree_gini(SPLITTED_DATA[1][:, -1]))/len(LABEL_FIT)
            DTREE_INFO['IMPORTANCES'][OPTIMAL_MODEL[0]] +=
(len(LABEL_FIT)/DTREE_INFO['n_data'])*gain
            DTREE_INFO['FEATURE_FREQUENCY'][OPTIMAL_MODEL[0]] += 1
        return SUB_TREE

def dtree_element(FEATURES_ELEMENT, TREE):
    branching = list(TREE.keys())[0]
    BRANCHING_PARAM = branching.split()
    if FEATURES_ELEMENT[int(BRANCHING_PARAM[0])] <= float(BRANCHING_PARAM[2]):
        BRANCH = TREE[branching][0]
    else:
        BRANCH = TREE[branching][1]
    if not isinstance(BRANCH, dict):
        return BRANCH[0]
    else:

```

```

        return dtree_element(FEATURES_ELEMENT, BRANCH)

def dtree_model(FEATURES_MODEL, TREE_MODEL):
    LABEL_MODEL = np.zeros(FEATURES_MODEL.shape[0])
    for j in range(FEATURES_MODEL.shape[0]):
        LABEL_MODEL[j] = dtree_element(FEATURES_MODEL[j], TREE_MODEL)
    return LABEL_MODEL

def dtree_benchmark(FEATURES_BENCHMARK, LABEL_BENCHMARK, LABEL_CHECK, TREE_CHECK):
    UNIQUE_CHECK = [[], []]
    UNIQUE_CHECK[0], UNIQUE_CHECK[1] = np.unique(LABEL_CHECK, return_counts = True)
    leaf = UNIQUE_CHECK[0][np.argmax(UNIQUE_CHECK[1])]
    error_leaf = len(LABEL_BENCHMARK[LABEL_BENCHMARK != leaf])
    error_node = len(LABEL_BENCHMARK[LABEL_BENCHMARK != dtree_model(FEATURES_BENCHMARK,
TREE_CHECK)])
    if error_leaf <= error_node:
        return [leaf, 'tree_pruned']
    else:
        return TREE_CHECK

def dtree_prune(FEATURES_PRUNE, LABEL_PRUNE, FEATURES_ACTUAL, LABEL_ACTUAL, TREE_PRUNE):
    pruning = list(TREE_PRUNE.keys())[0]
    BRANCH_PRUNE = [[], []]
    BRANCH_PRUNE[0], BRANCH_PRUNE[1] = TREE_PRUNE[pruning]
    if not isinstance(BRANCH_PRUNE[0], dict) and not isinstance(BRANCH_PRUNE[1], dict):
        return dtree_benchmark(FEATURES_ACTUAL, LABEL_ACTUAL, LABEL_PRUNE, TREE_PRUNE)
    else:
        PRUNING_PARAM = pruning.split()
        DATA_PRUNE = np.column_stack([FEATURES_PRUNE, LABEL_PRUNE])
        DATA_ACTUAL = np.column_stack([FEATURES_ACTUAL, LABEL_ACTUAL])
        CORRECT_PRUNE = DATA_PRUNE[DATA_PRUNE[:, int(PRUNING_PARAM[0])] <=
float(PRUNING_PARAM[2])]
        INCORRECT_PRUNE = DATA_PRUNE[DATA_PRUNE[:, int(PRUNING_PARAM[0])] >
float(PRUNING_PARAM[2])]
        CORRECT_ACTUAL = DATA_ACTUAL[DATA_ACTUAL[:, int(PRUNING_PARAM[0])] <=
float(PRUNING_PARAM[2])]
        INCORRECT_ACTUAL = DATA_ACTUAL[DATA_ACTUAL[:, int(PRUNING_PARAM[0])] >
float(PRUNING_PARAM[2])]
        if isinstance(BRANCH_PRUNE[0], dict):
            CORRECT = dtree_prune(CORRECT_PRUNE[:, :-1], CORRECT_PRUNE[:, -1],
CORRECT_ACTUAL[:, :-1], CORRECT_ACTUAL[:, -1], BRANCH_PRUNE[0])
        if isinstance(BRANCH_PRUNE[1], dict):
            INCORRECT = dtree_prune(INCORRECT_PRUNE[:, :-1], INCORRECT_PRUNE[:, -1],
INCORRECT_ACTUAL[:, :-1], INCORRECT_ACTUAL[:, -1], BRANCH_PRUNE[1])
        TREE_PRUNE = {pruning: [CORRECT, INCORRECT]}
        return dtree_benchmark(FEATURES_ACTUAL, LABEL_ACTUAL, LABEL_PRUNE, TREE_PRUNE)

def randfor_boot(FEATURES_BOOT, LABEL_BOOT, boot):
    if boot == None:
        boot = len(LABEL_BOOT)
    INDICES_BOOT = np.random.randint(low = 0, high = len(LABEL_BOOT), size = boot)
    DATA_BOOT = [[], []]
    DATA_BOOT[0] = FEATURES_BOOT[INDICES_BOOT]
    DATA_BOOT[1] = LABEL_BOOT[INDICES_BOOT]
    return DATA_BOOT

def randfor_fit(FEATURES_FIT, LABEL_FIT, n_trees = 100, forest_branches = 1E5,
forest_minsamples = 2, n_boot = None, n_features = None, return_trees = False):
    global RANDFOR_INFO
    RANDFOR_INFO = {}
    RANDFOR_INFO['IMPORTANCES'] = np.zeros(FEATURES_FIT.shape[1])
    if n_features == None:
        n_features = FEATURES_FIT.shape[1]

```

```

FOREST = []
for n in range(n_trees):
    [FEATURES_BOOTED, LABEL_BOOTED] = randfor_boot(FEATURES_FIT, LABEL_FIT, n_boot)
    TREE = dtree_fit(FEATURES_BOOTED, LABEL_BOOTED, branch_limit = forest_branches,
min_samples = forest_minsamples, n_subspace = n_features)
    FOREST.append(TREE)
    RANDFOR_INFO['IMPORTANCES'] += DTREE_INFO['IMPORTANCES']
    if return_trees:
        print(TREE)
return FOREST

def randfor_element(FEATURES_ELEMENT, FOREST_TREE):
    FOREST_RESULT = np.ones(len(FOREST_TREE))
    for n in range(len(FOREST_TREE)):
        FOREST_RESULT[n] = dtree_element(FEATURES_ELEMENT, TREE = FOREST_TREE[n])
    RESULT_UNIQUE = [], []
    RESULT_UNIQUE[0], RESULT_UNIQUE[1] = np.unique(FOREST_RESULT, return_counts = True)
    result_mode = RESULT_UNIQUE[0][np.argmax(RESULT_UNIQUE[1])]
    return result_mode

def randfor_model(FEATURES_MODEL, FOREST_MODEL):
    LABEL_MODEL = np.zeros(FEATURES_MODEL.shape[0])
    for j in range(FEATURES_MODEL.shape[0]):
        LABEL_MODEL[j] = randfor_element(FEATURES_MODEL[j], FOREST_MODEL)
    return LABEL_MODEL

# Data preprocessing
DF = DF.drop(columns = ['Tanggal', 'PCH Sarijadi', 'PCH Nanjung', 'PCH Cikapundung', 'PDA
Dayeuhkolot', 'PDA Baleendah'])
DF = DF.dropna()
DATA = DF.to_numpy()

train_ratio = 70E-2
TRAIN_FEATURES = DATA[:int(len(DF)*train_ratio), :-1]
TRAIN_LABEL = DATA[:int(len(DF)*train_ratio), -1]
TEST_FEATURES = DATA[int(len(DF)*train_ratio):, :-1]
TEST_LABEL = DATA[int(len(DF)*train_ratio):, -1]

# Data upsampling
# TRAIN_DATA = data_upsample(TRAIN_FEATURES, TRAIN_LABEL)
# TRAIN_FEATURES = TRAIN_DATA[:, :-1]
# TRAIN_LABEL = TRAIN_DATA[:, -1]

# Data overview
TRAIN_UNIQUE = []
TRAIN_OVERVIEW = [DF.iloc[:, :-1].columns.to_list(), []]
TEST_UNIQUE = []
TEST_OVERVIEW = [DF.iloc[:, :-1].columns.to_list(), []]
for i in range(TRAIN_FEATURES.shape[1]):
    TRAIN_UNIQUE.append(np.unique(TRAIN_FEATURES[:, i]))
    TRAIN_OVERVIEW[1].append(len(TRAIN_UNIQUE[i]))
    TEST_UNIQUE.append(np.unique(TEST_FEATURES[:, i]))
    TEST_OVERVIEW[1].append(len(TEST_UNIQUE[i]))

fig = plt.figure(figsize = (16, 6))
fig.suptitle("Features Variation Overview")
ax = fig.subplots(1, 2)

ax[0].barh(TRAIN_OVERVIEW[0], TRAIN_OVERVIEW[1])
ax[0].set_title("Training Data Features")
ax[0].set_xlabel("Feature variation amount")
ax[0].grid()

```

```

ax[1].barh(TEST_OVERVIEW[0], TEST_OVERVIEW[1])
ax[1].set_title("Testing Data Features")
ax[1].set_xlabel("Feature variation amount")
ax[1].grid()

plt.tight_layout()
plt.show()

TRAIN_POP = [], []
TRAIN_POP[0], TRAIN_POP[1] = np.unique(TRAIN_LABEL, return_counts = True)
TEST_POP = [], []
TEST_POP[0], TEST_POP[1] = np.unique(TEST_LABEL, return_counts = True)

fig = plt.figure(figsize = (16, 6))
fig.suptitle("Label population overview")
ax = fig.subplots(1, 2)

ax[0].bar(TRAIN_POP[0], TRAIN_POP[1])
ax[0].set_title("Training Data Labels")
ax[0].set_ylabel("Label population")
ax[0].grid()

ax[1].bar(TEST_POP[0], TEST_POP[1])
ax[1].set_title("Testing Data Labels")
ax[1].set_ylabel("Label population")
ax[1].grid()

plt.tight_layout()
plt.show()

# Linear regression modelling
# model parameters: slope_calc (fit_intercept in sklearn) and force_nn (positive in sklearn)
LR_MODEL = linreg_fit(TRAIN_FEATURES, TRAIN_LABEL, force_nn = True)
LR_FEATURES = [DF.iloc[:, :-1].columns.to_list(),
               ((LR_MODEL[0]/LR_MODEL[0].sum())*100).to_list()]
LR_FEATURES = list_sort(LR_FEATURES)

# Sklearn model for comparison
LR_MODEL_SKLEARN = LinearRegression(positive = True)
LR_MODEL_SKLEARN.fit(TRAIN_FEATURES, TRAIN_LABEL)
LR_FEATURES_SKLEARN = [DF.iloc[:, :-1].columns.to_list(),
                       ((LR_MODEL_SKLEARN.coef_/LR_MODEL_SKLEARN.coef_.sum())*100).to_list()]
LR_FEATURES_SKLEARN = list_sort(LR_FEATURES_SKLEARN)

# Plot linear regression model feature mportances
fig = plt.figure(figsize = (16, 6))
fig.suptitle("Feature Importance Model Linear Regression")
ax = fig.subplots(1, 2)

ax[0].barh(LR_FEATURES[0], LR_FEATURES[1])
ax[0].text(0, 0, "RMSE : %f" % root_mean_squared_error(TEST_LABEL,
linreg_model(TEST_FEATURES, LR_MODEL[0], LR_MODEL[1])))
ax[0].set_title("Linear Regression")
ax[0].set_xlabel("Importance (%)")
ax[0].grid()

ax[1].barh(LR_FEATURES_SKLEARN[0], LR_FEATURES_SKLEARN[1])
ax[1].text(0, 0, "RMSE : %f" % root_mean_squared_error(TEST_LABEL,
LR_MODEL_SKLEARN.predict(TEST_FEATURES)))
ax[1].set_title("Linear Regression Sklearn")
ax[1].set_xlabel("Importance (%)")
ax[1].grid()

```

```

plt.tight_layout()
plt.show()

# Plot model comparison
plt.scatter(LR_MODEL_SKLEARN.predict(TEST_FEATURES), linreg_model(TEST_FEATURES,
LR_MODEL[0], LR_MODEL[1]))
plt.text(0, 0, "Manual vs Sklearn Model RMSE : %f" %
root_mean_squared_error(LR_MODEL_SKLEARN.predict(TEST_FEATURES),
linreg_model(TEST_FEATURES, LR_MODEL[0], LR_MODEL[1])))
plt.title("Manual vs Sklearn Linear Model")
plt.xlabel("Sklearn Model")
plt.ylabel("Manual Model")
plt.grid()
plt.tight_layout()
plt.show()

# Decision tree modelling
DT_MODEL = dtree_fit(TRAIN_FEATURES, TRAIN_LABEL, branch_limit = 12, min_samples = 8)
DT_FEATURES = [DF.iloc[:, :-1].columns.tolist(),
((DTREE_INFO['IMPORTANCES']/DTREE_INFO['IMPORTANCES'].sum())*100).tolist()]
DT_FEATURES = list_sort(DT_FEATURES)
RESULT_LABEL = dtree_model(TEST_FEATURES, DT_MODEL)
class_accuracy(RESULT_LABEL, TEST_LABEL, title = "Manual Model Accuracy :")

# sklearn model for comparison
DT_MODEL_SKLEARN = DecisionTreeClassifier(max_depth = 12, min_samples_split = 8)
DT_MODEL_SKLEARN.fit(TRAIN_FEATURES, TRAIN_LABEL)
DT_FEATURES_SKLEARN = [DF.iloc[:, :-1].columns.tolist(),
(DT_MODEL_SKLEARN.feature_importances_*100).tolist()]
DT_FEATURES_SKLEARN = list_sort(DT_FEATURES_SKLEARN)
RESULT_LABEL_SKLEARN = DT_MODEL_SKLEARN.predict(TEST_FEATURES)
class_accuracy(RESULT_LABEL_SKLEARN, TEST_LABEL, title = "Sklearn Model Accuracy :")

# Show model comparison
RESULT_LABEL = dtree_model(TEST_FEATURES, DT_MODEL)
RESULT_LABEL_SKLEARN = DT_MODEL_SKLEARN.predict(TEST_FEATURES)
COMPARISON_TABLE = np.zeros((len(np.unique(RESULT_LABEL)),
len(np.unique(RESULT_LABEL_SKLEARN))))
for j in range(len(RESULT_LABEL)):
    COMPARISON_TABLE[int(RESULT_LABEL[j]), int(RESULT_LABEL_SKLEARN[j])] += 1

COMPARISON_TABLE = COMPARISON_TABLE/len(RESULT_LABEL)
COMPARISON_TABLE = pd.DataFrame(data = COMPARISON_TABLE, index = np.unique(RESULT_LABEL),
columns = np.unique(RESULT_LABEL_SKLEARN))
print(COMPARISON_TABLE)
print("Manual vs Sklearn Model RMSE : %f" %
root_mean_squared_error(DT_MODEL_SKLEARN.predict(TEST_FEATURES), dtree_model(TEST_FEATURES,
DT_MODEL)))

# Plot decision tree model feature importances
fig = plt.figure(figsize = (16, 6))
fig.suptitle("Feature Importance Model Decision Tree")
ax = fig.subplots(1, 2)

ax[0].barh(DT_FEATURES[0], DT_FEATURES[1])
ax[0].set_title("Decision Tree")
ax[0].set_xlabel("Importance (%)")
ax[0].grid()

ax[1].barh(DT_FEATURES_SKLEARN[0], DT_FEATURES_SKLEARN[1])
ax[1].set_title("Decision Tree Sklearn")
ax[1].set_xlabel("Importance (%)")
ax[1].grid()

```

```

plt.tight_layout()
plt.show()

# Plot all feature importances
fig = plt.figure(figsize = (8, 12))
fig.suptitle("Feature Importance")
ax = fig.subplots(2, 1)

ax[0].barh(LR_FEATURES[0], LR_FEATURES[1])
ax[0].text(0, 0, "RMSE : %f" % root_mean_squared_error(TEST_LABEL,
linreg_model(TEST_FEATURES, LR_MODEL[0], LR_MODEL[1])))
ax[0].set_title("Linear Regression")
ax[0].set_xlabel("Importance (%)")
ax[0].grid()

ax[1].barh(DT_FEATURES[0], DT_FEATURES[1])
ax[1].text(0, 0, "RMSE : %f" % root_mean_squared_error(TEST_LABEL, RESULT_LABEL))
ax[1].set_title("Decision Tree")
ax[1].set_xlabel("Importance (%)")
ax[1].grid()

plt.tight_layout()
plt.show()

# Show decision trees
pp.pprint(DT_MODEL)
print(export_text(DT_MODEL_SKLEARN))

```